

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 25 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 2D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89 //
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
106 //
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 //   memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116 //
117 #include <algorithm>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define WARP_SIZE 32
129 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
130 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
131 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
132
133 // =====
134 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
135 // =====
136
137 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
138 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
139 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
140 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
141 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
142 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
143 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
144 //
145 // 初始: 每个 lane 持有自己的值 v0..v7
146 // lane:  0    1    2    3    4    5    6    7
147 // val:  v0   v1   v2   v3   v4   v5   v6   v7
148 //
149 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
150 //
151 //      ┌──────────┐
152 // 对:   (0,4) (1,5) (2,6) (3,7)
153 //
154 // lane:  0    1    2    3    4    5    6    7
155 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
156 //
157 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
158 //
159 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
160 // 对:   (0,2) (1,3) (4,6) (5,7)
161 //      ─── 前4个一组 ───      ─── 后4个一组 ───
162 //
163 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
164 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
165 //      = v0+v2+v4+v6 ... (逐步归约)
166 //
167 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
168 //
169 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
170 // 对:   (0,1) (2,3) (4,5) (6,7)
171 //
172 // lane:  0    1    2    3    4    5    6    7
173 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
174 //
175 // mask=0: 循环终止, 归约完成。
176 //
177 // 关键性质:
178 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
179 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
180 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
181 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
182 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
183 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
184 template <const int kWarpSize = WARP_SIZE, typename T = float>
185 __device__ __forceinline__ T warp_reduce_sum(T val) {
186 #pragma unroll
187   for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
188     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
189   }
190   return val;
191 }

```

```

189
190 // Warp Reduce Max — generic
191 template <const int kWarpSize = WARP_SIZE, typename T = float>
192 __device__ __forceinline__ T warp_reduce_max(T val) {
193 #pragma unroll
194     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
195         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
196     }
197     return val;
198 }
199
200 // =====
201 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
202 // =====
203
204 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
205 // 两级归约流程:
206 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
207 // 2. warp leader (lane=0) 写入 shared memory
208 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
209 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
210 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
211 template <const int NUM_THREADS = 256>
212 __device__ float block_reduce_sum(float val) {
213     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
214     int warp = threadIdx.x / WARP_SIZE;
215     int lane = threadIdx.x % WARP_SIZE;
216     static __shared__ float shared[NUM_WARPS];
217
218     float value = warp_reduce_sum<WARP_SIZE>(val);
219     if (lane == 0)
220         shared[warp] = value;
221     __syncthreads();
222     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
223     value = warp_reduce_sum<NUM_WARPS>(value);
224     // 关键: broadcast 结果到所有线程, 后续用 result
225     // 做除法等操作时所有线程都能拿到
226     value = __shfl_sync(0xffffffff, value, 0, 32);
227     return value;
228 }
229
230 // Block Reduce Max — FP32 (增强版, 带 broadcast)
231 template <const int NUM_THREADS = 256>
232 __device__ float block_reduce_max(float val) {
233     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
234     int warp = threadIdx.x / WARP_SIZE;
235     int lane = threadIdx.x % WARP_SIZE;
236     static __shared__ float shared[NUM_WARPS];
237
238     float value = warp_reduce_max<WARP_SIZE>(val);
239     if (lane == 0)
240         shared[warp] = value;
241     __syncthreads();
242     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
243     value = warp_reduce_max<NUM_WARPS>(value);
244     value = __shfl_sync(0xffffffff, value, 0, 32);
245     return value;
246 }
247
248 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
249 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
250 // 跨 block 求和的常见模式, 适合 N 较大时使用;
251 // Grid: ((N + 255) / 256, 1, 1)

```

```

252 // Block: (256, 1, 1)
253 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
254 template <const int NUM_THREADS = 256>
255 __global__ void block_reduce_all(float *a, float *y, int N) {
256     int tid = threadIdx.x;
257     int idx = blockIdx.x * NUM_THREADS + tid;
258     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
259     __shared__ float reduce_smem[NUM_WARPS];
260
261     float val = (idx < N) ? a[idx] : 0.0f;
262     int warp = tid / WARP_SIZE;
263     int lane = tid % WARP_SIZE;
264
265     val = warp_reduce_sum<WARP_SIZE>(val);
266     if (lane == 0)
267         reduce_smem[warp] = val;
268     __syncthreads();
269
270     val = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
271     if (warp == 0)
272         val = warp_reduce_sum<NUM_WARPS>(val);
273     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
274         atomicAdd(y, val);
275 }
276
277 // ---- Dot Product: y = sum(a[i] * b[i]) ----
278 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
279 // Grid: ((N + 255) / 256, 1, 1)
280 // Block: (256, 1, 1)
281 // source: LeetCUDA/kernels/dot-product/dot_product.cu
282 template <const int NUM_THREADS = 256>
283 __global__ void dot(float *a, float *b, float *y, int N) {
284     int tid = threadIdx.x;
285     int idx = blockIdx.x * NUM_THREADS + tid;
286     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
287     __shared__ float reduce_smem[NUM_WARPS];
288
289     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
290     int warp = tid / WARP_SIZE;
291     int lane = tid % WARP_SIZE;
292
293     prod = warp_reduce_sum<WARP_SIZE>(prod);
294     if (lane == 0)
295         reduce_smem[warp] = prod;
296     __syncthreads();
297
298     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
299     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
300         prod = warp_reduce_sum<NUM_WARPS>(prod);
301     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
302         atomicAdd(y, prod);
303 }
304
305 // Dot Product + float4
306 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
307 // Block: (64, 1, 1), 256/4=64
308 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
309 // source: LeetCUDA/kernels/dot-product/dot_product.cu
310 template <const int NUM_THREADS = 256 / 4>
311 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
312     int tid = threadIdx.x;
313     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
314     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;

```

```

315 __shared__ float reduce_smem[NUM_WARPS];
316
317 float4 reg_a = FLOAT4(a[idx]);
318 float4 reg_b = FLOAT4(b[idx]);
319 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
320                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
321                  : 0.0f;
322 int warp = tid / WARP_SIZE;
323 int lane = tid % WARP_SIZE;
324
325 prod = warp_reduce_sum<WARP_SIZE>(prod);
326 if (lane == 0)
327     reduce_smem[warp] = prod;
328 __syncthreads();
329
330 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
331 if (warp == 0)
332     prod = warp_reduce_sum<NUM_WARPS>(prod);
333 if (tid == 0)
334     atomicAdd(y, prod);
335 }
336
337
338 // =====
339 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
340 // =====
341 // 面试要点:
342 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
343 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
344 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
345
346 // ---- ReLU: y = max(0, x) ----
347 // Grid: ((N + 255) / 256, 1, 1)
348 // Block: (256, 1, 1)
349 // source: LeetCUDA/kernels/relu/relu.cu
350 __global__ void relu(float *x, float *y, int N) {
351     int idx = blockIdx.x * blockDim.x + threadIdx.x;
352     if (idx < N)
353         y[idx] = fmaxf(0.0f, x[idx]);
354 }
355
356 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
357 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
358 // Grid: ((N + 255) / 256, 1, 1)
359 // Block: (64, 1, 1)
360 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
361 // source: LeetCUDA/kernels/relu/relu.cu
362 __global__ void relu_vec4(float *x, float *y, int N) {
363     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
364     if (idx < N) {
365         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
366         float4 reg_y;
367         reg_y.x = fmaxf(0.0f, reg_x.x);
368         reg_y.y = fmaxf(0.0f, reg_x.y);
369         reg_y.z = fmaxf(0.0f, reg_x.z);
370         reg_y.w = fmaxf(0.0f, reg_x.w);
371         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
372     }
373 }
374
375 // ---- Elementwise Add: c = a + b ----
376 // Grid: ((N + 255) / 256, 1, 1)
377 // Block: (256, 1, 1)

```

```

378 // source: LeetCUDA/kernels/elementwise/elementwise.cu
379 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
380     int idx = blockIdx.x * blockDim.x + threadIdx.x;
381     if (idx < N)
382         c[idx] = a[idx] + b[idx];
383 }
384
385 // Elementwise Add + float4 向量化
386 // Grid: ((N + 255) / 256, 1, 1)
387 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
388 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
389 // source: LeetCUDA/kernels/elementwise/elementwise.cu
390 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
391     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
392     if ((idx + 3) < N) {
393         float4 reg_a = FLOAT4(a[idx]);
394         float4 reg_b = FLOAT4(b[idx]);
395         float4 reg_c;
396         reg_c.x = reg_a.x + reg_b.x;
397         reg_c.y = reg_a.y + reg_b.y;
398         reg_c.z = reg_a.z + reg_b.z;
399         reg_c.w = reg_a.w + reg_b.w;
400         FLOAT4(c[idx]) = reg_c;
401     } else if (idx < N) {
402         for (int i = 0; (idx + i) < N; i++) {
403             c[idx + i] = a[idx + i] + b[idx + i];
404         }
405     }
406 }
407
408 // ---- Histogram: y[a[i]]++ ----
409 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
410 // Grid: ((N + 255) / 256, 1, 1)
411 // Block: (256, 1, 1)
412 // source: LeetCUDA/kernels/histogram/histogram.cu
413 __global__ void histogram(int *a, int *y, int N) {
414     int idx = blockIdx.x * blockDim.x + threadIdx.x;
415     if (idx < N)
416         atomicAdd(&(y[a[idx]]), 1);
417 }
418
419 // =====
420 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
421 // =====
422 // 面试要点:
423 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
424 //     online (1-pass, 增量更新)
425 //   - Online Softmax 是 FlashAttention 的数学基础
426 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
427 //     + variance)
428 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
429
430 // =====
431 // Phase 3a: Softmax — 三级递进 (面试核心考点)
432 // =====
433 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
434 // 回答线索: naive(溢出) → safe → online
435
436 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
437 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
438 // 问题: x 值很大时 exp(x) 溢出为 inf
439 template <const int NUM_THREADS = 256>
440 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL

```



```

441 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
442 // source: LeetCode/kernels/softmax/softmax.cu
443 __global__ void softmax_per_token(float *x, float *y, int N) {
444     const int tid = threadIdx.x;
445     const int idx = blockIdx.x * blockDim.x + tid;
446
447     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
448     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
449     if (idx < N)
450         y[idx] = exp_val / exp_sum;
451 }
452
453 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
454 // 面试重点: 为什么 Softmax 需要 Safe?
455 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
456 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
457 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
458 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
459 // Grid: (S, 1, 1)
460 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
461 // source: LeetCode/kernels/softmax/softmax.cu
462 template <const int NUM_THREADS = 256>
463 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
464     const int tid = threadIdx.x;
465     const int idx = blockIdx.x * blockDim.x + tid;
466
467     // Pass 1: block reduce max — 找最大值
468     float val = (idx < N) ? x[idx] : -FLT_MAX;
469     float max_val = block_reduce_max<NUM_THREADS>(val);
470
471     // Pass 2: exp(x - max) → block reduce sum
472     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
473     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
474
475     if (idx < N)
476         y[idx] = exp_val / exp_sum;
477 }
478
479 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
480 // 面试重点:
481 // 两种使用场景 (公式不同, 但结果等价):
482 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
483 //       m_new = max(m_old, x_i)
484 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
485 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
486 //       m = max(m1, m2)
487 //       d = d1*exp(m1-m) + d2*exp(m2-m)
488 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
489 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
490 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
491 //
492 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
493 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
494 //   (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
495 //   (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2
496 //
497 // 合并公式 (对称, 不依赖"新/旧"概念):
498 //   m = max(m1, m2)
499 //   d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
500 //
501 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
502 struct __align__(8) MD {
503     float m; // running max

```

```

504     float d; // running denominator (sum of exp(x - max))
505 };
506
507 template <const int kWarpSize = WARP_SIZE>
508 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
509     #pragma unroll
510     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
511         MD md2;
512         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpSize);
513         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpSize);
514
515         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
516         MD s_m = (md1.m > md2.m) ? md2 : md1;
517
518         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
519         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
520         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
521         md1.m = b_m.m;
522     }
523     return md1;
524 }
525
526 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
527 // Grid: (S, 1, 1)
528 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
529 // source: LeetCUDA/kernels/softmax/softmax.cu
530 template <const int NUM_THREADS = 256>
531 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
532     int tid = threadIdx.x;
533     int idx = blockIdx.x * NUM_THREADS + threadIdx.x;
534     const int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
535     int warp = tid / WARP_SIZE;
536     int lane = tid % WARP_SIZE;
537
538     // 初始化: 每个线程持有一个 (max, denom) 对
539     MD md;
540     md.m = idx < N ? x[idx] : -FLT_MAX;
541     md.d = idx < N ? 1.0f : 0.0f;
542
543     // Block reduce: warp_reduce_md 在归约中自动更新 m 和 d
544     __shared__ MD shared[NUM_WARPS];
545     md = warp_reduce_md<WARP_SIZE>(md);
546
547     if (lane == 0)
548         shared[warp] = md;
549     __syncthreads();
550
551     // 第二级归约: warp0 收集各 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
552     // 只有 tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
553     if (tid < WARP_SIZE) {
554         md = tid < NUM_WARPS ? shared[tid] : MD{-FLT_MAX, 0.0f};
555         md = warp_reduce_md<NUM_WARPS>(md);
556         if (tid == 0) {
557             shared[0] = md;
558         }
559     }
560     __syncthreads();
561
562     // 用全局 max 和 denom 做最终 softmax
563     md = shared[0];
564     float d_inv = __fdividef(1.0f, md.d);
565     // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
566     if (idx < N) {

```

```

567     y[idx] = __expf(x[idx] - md.m) * d_inv;
568 }
569 }
570
571 // =====
572 // Phase 3b: RMS Normalization (1-pass reduce)
573 // =====
574 // 面试要点:
575 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
576 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
577 //   - Llama 系列使用 RMS Norm
578 //   - grid(N, K/K), block(K): 一行一个 block
579 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
580 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
581 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
582 template <const int NUM_THREADS = 128>
583 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
584     int tid = threadIdx.x;
585     int idx = blockIdx.x * blockDim.x + threadIdx.x;
586     const float epsilon = 1e-5f;
587
588     __shared__ float s_variance;
589     float value = (idx < N * K) ? x[idx] : 0.0f;
590     float variance = value * value;
591     variance = block_reduce_sum<NUM_THREADS>(variance);
592     if (tid == 0)
593         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
594     __syncthreads();
595     if (idx < N * K)
596         y[idx] = (value * s_variance) * g;
597 }
598
599 // RMS Norm + float4
600 // Grid: (N, 1, 1)
601 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
602 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
603 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
604 template <const int NUM_THREADS = 128 / 4>
605 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
606     int tid = threadIdx.x;
607     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
608     const float epsilon = 1e-5f;
609
610     __shared__ float s_variance;
611     float4 reg_x = FLOAT4(x[idx]);
612     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
613                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
614                             : 0.0f;
615     variance = block_reduce_sum<NUM_THREADS>(variance);
616     if (tid == 0)
617         s_variance = rsqrtf(variance / (float)K + epsilon);
618     __syncthreads();
619     float4 reg_y;
620     reg_y.x = reg_x.x * s_variance * g;
621     reg_y.y = reg_x.y * s_variance * g;
622     reg_y.z = reg_x.z * s_variance * g;
623     reg_y.w = reg_x.w * s_variance * g;
624     if (idx < N * K)
625         FLOAT4(y[idx]) = reg_y;
626 }
627
628 // =====
629 // Phase 3c: Layer Normalization (2-pass reduce)

```

```

630 // =====
631 // 面试要点:
632 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
633 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)^2)/K)
634 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
635 // Grid: (N, 1, 1), 一行一个 block
636 // Block: (128, 1, 1), NUM_THREADS=K=128
637 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
638 template <const int NUM_THREADS = 128>
639 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
640     int tid = threadIdx.x;
641     int idx = blockIdx.x * blockDim.x + threadIdx.x;
642     const float epsilon = 1e-5f;
643
644     __shared__ float s_mean;
645     __shared__ float s_variance;
646     float value = (idx < N * K) ? x[idx] : 0.0f;
647
648     // Pass 1: compute mean
649     float sum = block_reduce_sum<NUM_THREADS>(value);
650     if (tid == 0)
651         s_mean = sum / (float)K;
652     __syncthreads(); // 必须等待 s_mean 对所有线程可见
653
654     // Pass 2: compute variance = (x - mean)^2
655     float variance = (value - s_mean) * (value - s_mean);
656     variance = block_reduce_sum<NUM_THREADS>(variance);
657     if (tid == 0)
658         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
659     __syncthreads(); // 必须等待 s_variance 对所有线程可见
660
661     if (idx < N * K)
662         y[idx] = ((value - s_mean) * s_variance) * g + b;
663 }
664
665 // Layer Norm + float4
666 // Grid: (N, 1, 1)
667 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
668 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
669 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
670 template <const int NUM_THREADS = 128 / 4>
671 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
672     int tid = threadIdx.x;
673     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
674     const float epsilon = 1e-5f;
675
676     __shared__ float s_mean;
677     __shared__ float s_variance;
678     float4 reg_x = FLOAT4(x[idx]);
679     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
680
681     float sum = block_reduce_sum<NUM_THREADS>(value);
682     if (tid == 0)
683         s_mean = sum / (float)K;
684     __syncthreads();
685
686     float4 reg_x_hat;
687     reg_x_hat.x = reg_x.x - s_mean;
688     reg_x_hat.y = reg_x.y - s_mean;
689     reg_x_hat.z = reg_x.z - s_mean;
690     reg_x_hat.w = reg_x.w - s_mean;
691     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
692                     reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;

```

```

693 variance = block_reduce_sum<NUM_THREADS>(variance);
694 if (tid == 0)
695     s_variance = rsqrtf(variance / (float)K + epsilon);
696 __syncthreads();
697
698 float4 reg_y;
699 reg_y.x = reg_x_hat.x * s_variance * g + b;
700 reg_y.y = reg_x_hat.y * s_variance * g + b;
701 reg_y.z = reg_x_hat.z * s_variance * g + b;
702 reg_y.w = reg_x_hat.w * s_variance * g + b;
703 if (idx < N * K)
704     FLOAT4(y[idx]) = reg_y;
705 }
706
707 // =====
708 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
709 // =====
710 // 面试要点:
711 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
712 //     [x1']   [cos(θ)  -sin(θ)] [x1]
713 //     [x2'] = [sin(θ)   cos(θ)] [x2]
714 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
715 //   - token_pos = idx / N: token 在序列中的位置
716 //   - token_idx = idx % N: token 内的维度索引
717 //   - 输入 [seq_len, hidden_size], 输出同形状
718
719 // Grid: ((seq_len * N + 255) / 256, 1, 1)
720 // Block: (256, 1, 1)
721 // source: LeetCUDA/kernels/rope/rope.cu
722 __global__ void rope(float *x, float *out, int seq_len, int N) {
723     int idx = blockIdx.x * blockDim.x + threadIdx.x;
724     float x1 = x[idx * 2];
725     float x2 = x[idx * 2 + 1];
726     int token_pos = idx / N; // 序列位置
727     int token_idx = idx % N; // 维度索引
728
729     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
730     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
731     float sin_v = sinf(token_pos * exp_v);
732     float cos_v = cosf(token_pos * exp_v);
733
734     // 2D 旋转
735     float out1 = x1 * cos_v - x2 * sin_v;
736     float out2 = x1 * sin_v + x2 * cos_v;
737     out[idx * 2] = out1;
738     out[idx * 2 + 1] = out2;
739 }
740
741 // =====
742 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
743 // =====
744 // 面试要点 (Bank Conflict 专题):
745 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
746 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
747 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
748 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
749 //
750 // 转置的四步演进:
751 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
752 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
753 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
754 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
755 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略

```

```

756 // 每个线程处理 1 个元素, block(16,16)
757 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
758 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
759 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
760 // Block: (16, 16, 1)
761 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
762 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
763     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
764     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
765     if (r < row && c < col) {
766         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
767         y[c * row + r] = x[r * col + c];
768     }
769 }
770 }
771
772 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
773 // Block: (16, 16, 1)
774 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
775 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
776 __global__ void mat_transpose_padded(
777     float *x, float *y, const int row, const int col) {
778     const int tx = threadIdx.x, ty = threadIdx.y;
779
780     constexpr int TILE = 16;
781     constexpr int PAD = 1;
782     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
783     __shared__ float tile[TILE * 4][TILE + PAD];
784
785     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
786     const int x_c = blockIdx.x * TILE + tx;
787     const int x_r = blockIdx.y * TILE + ty;
788
789     if (x_r * 4 < row && x_c < col) {
790         // Step 1: 4 rows × 1 col → smem (row-major);
791 #pragma unroll
792         for (int i = 0; i < 4; ++i) {
793             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
794         }
795         __syncthreads();
796
797         // Step 2: Transposed read from smem
798         float4 reg_trans;
799         reg_trans.x = tile[tx * 4 + 0][ty];
800         reg_trans.y = tile[tx * 4 + 1][ty];
801         reg_trans.z = tile[tx * 4 + 2][ty];
802         reg_trans.w = tile[tx * 4 + 3][ty];
803
804         // y 空间坐标: y_r = x 的列, y_c = x 的行
805         const int y_r = blockIdx.x * TILE + ty; // = x col
806         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
807
808         // Coalesced write: y[y_r][y_c .. y_c+3]
809         FLOAT4(y[y_r * row + y_c]) = reg_trans;
810     }
811 }
812
813 // =====
814 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
815 // =====
816 // 面试要点:
817 // - GEMV 是典型的 memory-bound 算子: AI ≈ O(1), 瓶颈在内存带宽
818 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)

```

```

819 // - 不同 K 值对应不同分块策略:
820 //     K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
821 //     K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
822 //     K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
823 //
824 // a: MxK, x: Kx1, y: Mx1, 计算: y = a * x; N = 1
825
826 // ---- SGEMV K32: 基础 warp-per-row ----
827 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
828 // grid(M/4), 每个 warp 负责一行
829 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
830 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
831 // Block: (32, 4, 1), 每行 1 warp
832 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
833 // source: LeetCUDA/kernels/sgemv/sgemv.cu
834 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
835     int tx = threadIdx.x; // 0~31
836     int ty = threadIdx.y; // 0~3
837     int lane = tx % WARP_SIZE; // 0~31
838     int m = blockIdx.x * blockDim.y + ty; // 全局行号
839     if (m < M) {
840         float sum = 0.0f;
841         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
842         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
843         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
844 #pragma unroll
845         for (int w = 0; w < NUM_ITERS; ++w) {
846             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
847             int k = w * WARP_SIZE + lane;
848             sum += a[m * K + k] * x[k];
849         }
850         sum = warp_reduce_sum<WARP_SIZE>(sum);
851         // 每个 warp 处理一行, lane 0 写回结果
852         if (lane == 0)
853             y[m] = sum;
854     }
855 }
856
857 // ---- SGEMV K128: float4 向量化 ----
858 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
859 // Grid: ((M + 3) / 4, 1, 1)
860 // Block: (32, 4, 1)
861 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
862 // source: LeetCUDA/kernels/sgemv/sgemv.cu
863 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
864     int tx = threadIdx.x; // 0~31
865     int ty = threadIdx.y; // 0~3
866     int lane = tx % WARP_SIZE; // 0~31
867     int m = blockDim.y * blockIdx.x + ty;
868
869     if (m < M) {
870         float sum = 0.0f;
871         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
872         const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
873 #pragma unroll
874         for (int w = 0; w < NUM_ITERS; ++w) {
875             int k = (w * WARP_SIZE + lane) * 4;
876             float4 reg_x = FLOAT4(x[k]);
877             float4 reg_a = FLOAT4(a[m * K + k]);
878             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
879                 reg_a.w * reg_x.w);
880         }
881         sum = warp_reduce_sum<WARP_SIZE>(sum);

```



```

882     if (lane == 0)
883         y[m] = sum;
884     }
885 }
886
887 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
888 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
889 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
890 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
891 // Block: (32, 4, 1)
892 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
893 // source: LeetCUDA/kernels/sgemv/sgemv.cu
894 template <const int ROW_PER_WARP = 2>
895 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
896     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
897     int tx = threadIdx.x;
898     int ty = threadIdx.y;
899     int lane = tx % WARP_SIZE;
900     int k = lane % K_WARP_SIZE; // 0~15
901     int m = (blockDim.y * blockIdx.x + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
902     if (m < M) {
903         float sum = A[m * K + k] * x[k];
904         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
905         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
906         // 注意: 判断条件是 k == 0, 不是 lane == 0!
907         if (k == 0)
908             y[m] = sum;
909     }
910 }
911
912 // =====
913 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
914 // =====
915 // 面试要点 (GEMM 优化五层金字塔):
916 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
917 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
918 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
919 //   load/store 指令数 Level 4 — Tensor Core (MMA
920 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
921 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
922 //
923 // 计算密度递进:
924 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
925 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
926 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
927 //
928 // =====
929 // Phase 7a: SGEMM + HGEMM (非 Tensor Core 路径)
930 // =====
931
932 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
933 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
934 // C = A × B, C[M, N] = A[M, K] × B[K, N]
935 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
936 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
937 // Block: (32, 32, 1), 1024 线程
938 // source: LeetCUDA/kernels/sgemm/sgemm.cu
939 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
940     constexpr int BM = 32; // vec 版: 32×4 = 128
941     constexpr int BN = 32; // vec 版: 32×4 = 128
942     constexpr int BK = 32;
943     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32×32×4=4KB smem, float = 4 bytes
944

```



```

945 int bx = blockIdx.x;
946 int by = blockIdx.y;
947 int tx = threadIdx.x;
948 int tid = threadIdx.y * blockDim.x + tx;
949
950 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
951 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
952 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
953 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
954 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
955 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
956 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
957
958 float sum = 0.f; // 遍历完整的K, slice K;
959 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
960     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
961     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
962     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
963     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
964     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
965     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
966     __syncthreads(); // 确保整个 smem tile 加载完毕
967
968 #pragma unroll
969     for (int k = 0; k < BK; ++k) {
970         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
971         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
972         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
973     }
974     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
975 }
976 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
977 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
978 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
979 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
980 }
981
982 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
983 // 在 Level 1 基础上引入两层优化:
984 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
985 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
986 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
987 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
988 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
989 // 1024 x 16 = 16384 = 128 x 128 ✓
990 //
991 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
992 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
993 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
994 //
995 // 加载映射 (每线程 4 个元素, float4):
996 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
997 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
998 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
999 //
1000 // △ Bank Conflict 提示 (面试加分点):
1001 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1002 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1003 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1004 //
1005 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1006 // Block: (32, 32, 1), 1024 线程
1007 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)

```

```

1008 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1009 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1010     constexpr int BM = 128;
1011     constexpr int BN = 128;
1012     constexpr int BK = 32;
1013     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1014     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1015
1016     int bx = blockIdx.x;
1017     int by = blockIdx.y;
1018     int tx = threadIdx.x;
1019     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1020
1021     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1022     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1023     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1024     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1025     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1026     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1027
1028     int load_gmem_a_m = by * BM + load_smem_a_m;
1029     int load_gmem_b_n = bx * BN + load_smem_b_n;
1030
1031     // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1032     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1033     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1034
1035     float sum[4][4] = {0.f};
1036     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1037         int load_gmem_a_k = bk * BK + load_smem_a_k;
1038         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1039         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1040         int load_gmem_b_k = bk * BK + load_smem_b_k;
1041         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1042         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1043         __syncthreads();
1044
1045     #pragma unroll
1046         for (int k = 0; k < BK; ++k) {
1047             // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1048             float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1049                                 s_a[comp_smem_a_m_base + 1][k],
1050                                 s_a[comp_smem_a_m_base + 2][k],
1051                                 s_a[comp_smem_a_m_base + 3][k]};
1052             float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1053                                 s_b[k][comp_smem_b_n_base + 1],
1054                                 s_b[k][comp_smem_b_n_base + 2],
1055                                 s_b[k][comp_smem_b_n_base + 3]};
1056
1057             #pragma unroll
1058             for (int i = 0; i < 4; ++i) {
1059                 #pragma unroll
1060                 for (int j = 0; j < 4; ++j) {
1061                     sum[i][j] += a_vals[i] * b_vals[j];
1062                 }
1063             }
1064             __syncthreads();
1065         }
1066
1067         // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1068         int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1069         int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1070     #pragma unroll

```

```

1071 for (int i = 0; i < 4; ++i) {
1072     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1073     float4 reg_c;
1074     reg_c.x = sum[i][0];
1075     reg_c.y = sum[i][1];
1076     reg_c.z = sum[i][2];
1077     reg_c.w = sum[i][3];
1078     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1079 }
1080 }
1081
1082 // =====
1083 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1084 // =====
1085 // 面试要点:
1086 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1087 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16×8] = [16×16]×[16×8]
1088 //   - ldmatrix: 从 shared memory 加载 16×16 的矩阵片段到寄存器 (4 条 32-bit
1089 //     寄存器)
1090 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1091 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1092 //
1093 // ★ TN 布局详解 (面试高频考点):
1094 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1095 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1096 //   N = Normal (列优先, BLAS 原生格式)
1097 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1098 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1099 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1100 //   BLAS=Normal) 即: C = op(A) × op(B) = AT × B? 不对! 在 row-major
1101 //   视角下: TN = A row-major [M×K], BT row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1102 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1103 //   T on A: BLAS 把 row-major 的 A 视为 AT, 传 T 表示"转置回去"
1104 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1105 //
1106 //   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1107 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1108 //   N = col-major (列优先, BLAS native = normal)
1109 //
1110 //   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1111 //   - A: row-major [M, K], B: row-major [K, N]
1112 //   - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1113 //   - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1114 //
1115 //   TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 BT=[N×K] row-major 存储 (A 行优先, B 列优先)
1116 //   - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1117 //   - BT [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1118 //   - 优势: BT 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1119 //   MMA 指令: mma.sync.aligned.m16n8k16.row.col
1120 //   - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1121 //
1122 //   WGMMMA (Warp Group MMA, Hopper+):
1123 //   - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1124 //   - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1125 //   - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1126 //     threads) 做计算
1127 //   - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1128 //
1129 //   =====
1130 //   Phase 7b-1: MMA PTX 宏定义
1131 //   =====
1132 //
1133 //   ---- gmem → smem: cp.async ----

```

```

1134 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1135 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1136 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1137 //     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1138 //   - wait_all: 等价于 commit_group + wait_group 0。
1139 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1140 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1141 #define CP_ASYNC_WAIT_GROUP(n) \
1142     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1143
1144 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1145 #define CP_ASYNC_CG(dst, src, bytes) \
1146     asm volatile( \
1147         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1148         "l"(src), "n"(bytes))
1149
1150 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1151 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1152 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1153 // aligned: 要求 128-bit 对齐, trans: 转置加载
1154 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1155     asm volatile( \
1156         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1157         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1158         : "r"(addr))
1159
1160 #define LDMATRIX_X2(R0, R1, addr) \
1161     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1162         : "=r"(R0), "=r"(R1) \
1163         : "r"(addr))
1164
1165 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1166 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1167 #define LDMATRIX_X2_T(R0, R1, addr) \
1168     asm volatile( \
1169         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1170         : "=r"(R0), "=r"(R1) \
1171         : "r"(addr))
1172
1173 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1174 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1175 // row.col: A 是 row-major, B 是 col-major
1176 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1177 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1178 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1179 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1180     asm volatile( \
1181         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1182         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1183         : "=r"(RD0), "=r"(RD1) \
1184         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1185         "r"(RC1))
1186
1187 // ---- MMA 辅助函数 ----
1188 // div_ceil: 整数除法向上取整
1189 #define HOST_DEVICE_INLINE __device__ __host__ inline
1190 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1191     return (a % b != 0) ? (a / b + 1) : (a / b);
1192 }
1193
1194 // =====
1195 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1196 // =====

```

```

1197 // 面试重点 — Tile Hierarchy:
1198 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1199 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1200 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1201 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, VAL_TILE_M=4, VAL_TILE_N=4
1202 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1203 //
1204 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1205 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1206 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1207 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1208 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1209 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1210 //
1211 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1212 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % K_STAGE (零偏移)
1213 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+K_STAGE-1) 供后续使用
1214 //   - cp.async 条件化: 仅当 k+K_STAGE-1 < NUM_K_TILES 时加载
1215 //   - WAIT_GROUP 自适应: 满载期用 K_STAGE-2, 尾部排空用 0
1216 //
1217 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1218 // Block: (256, 1, 1), 8 warps
1219 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1220 // =====
1221 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1222           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1223           const int VAL_TILE_M = 4, const int VAL_TILE_N = 4,
1224           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1225 __global__ void __launch_bounds__(256)
1226   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1227   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1228   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1229   const int by = blockIdx.y;
1230   constexpr int BM = MMA_M * MMA_TILE_M * VAL_TILE_M; // 16*2*4=128
1231   constexpr int BN = MMA_N * MMA_TILE_N * VAL_TILE_N; // 8*4*4=128
1232   constexpr int BK = MMA_K; // 16
1233
1234   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1235   // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1236   // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1237   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1238   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1239   extern __shared__ half smem[];
1240   half *s_a = smem;
1241   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1242   constexpr int s_a_stage_offset = BM * BK; // 128*16
1243   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major
1244   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1245   const int warp_id = tid / WARP_SIZE; // 0~7
1246   const int lane_id = tid % WARP_SIZE; // 0~31
1247   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1248   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1249
1250   // 线程到 global memory 的映射 (用于加载 A 和 B)
1251   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1252   int load_smem_a_m = tid / 2; // 0~127
1253   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1254   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1255   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1256   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1257   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1258   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1259     return;

```

```

1260
1261 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1262 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1263 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是VAL Tile的作用。
1264 uint32_t RC[VAL_TILE_M][VAL_TILE_N][2] = {0}; // 初始化为 0
1265
1266 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1267 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1268 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1269
1270 // 预加载前 (K_STAGE-1) 个 stage
1271 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1272 #pragma unroll
1273 for (int k = 0; k < (K_STAGE - 1); ++k) {
1274     int load_gmem_a_k = k * BK + load_smem_a_k;
1275     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1276     int load_gmem_b_k = k * BK + load_smem_b_k;
1277     // B^T: [n][k] row-major (即 B[k][n] col-major) Δ
1278     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1279
1280     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1281         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1282     );
1283     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1284
1285     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1286         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1287     );
1288     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1289
1290     CP_ASYNC.COMMIT_GROUP();
1291 }
1292
1293 const int NUM_K_TILES = div_ceil(K, BK);
1294 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 允许有 K_STAGE-2 个group未完成
1295 __syncthreads();
1296
1297 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1298 #pragma unroll
1299 for (int k = 0; k < NUM_K_TILES; ++k) {
1300     int smem_sel = k % K_STAGE; // 计算 tile k 的 stage
1301     int smem_sel_next = (k + K_STAGE - 1) % K_STAGE; // 预加载目标 stage
1302
1303     // 条件加载: 预加载 tile (k+K_STAGE-1) 供将来使用
1304     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k (row-major, 内维连续的是 K)
1305     if (k + K_STAGE - 1 < NUM_K_TILES) {
1306         int load_gmem_a_k = (k + K_STAGE - 1) * BK + load_smem_a_k;
1307         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1308         int load_gmem_b_k = (k + K_STAGE - 1) * BK + load_smem_b_k;
1309         // B^T: row-major [n][k], 内维连续的是 K Δ
1310         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1311
1312         uint32_t load_smem_a_ptr =
1313             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1314                 load_smem_a_m * BK + load_smem_a_k) *
1315                 sizeof(half));
1316         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1317
1318         uint32_t load_smem_b_ptr =
1319             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1320                 load_smem_b_n * BK + load_smem_b_k) *
1321                 sizeof(half));
1322         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);

```



```

1323 CP_ASYNC_COMMIT_GROUP();
1324 }
1325
1326 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1327 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1328 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1329 uint32_t RA[VAL_TILE_M][4];
1330 uint32_t RB[VAL_TILE_N][2];
1331
1332 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1333 #pragma unroll
1334 for (int i = 0; i < VAL_TILE_M; ++i) {
1335     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1336     int warp_smem_a_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1337     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1338     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1339     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1340     uint32_t lane_smem_a_ptr =
1341         (smem_a_base_ptr +
1342          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1343           sizeof(half));
1344     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1345 }
1346
1347 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1348 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1349 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1350 #pragma unroll
1351 for (int j = 0; j < VAL_TILE_N; ++j) {
1352     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1353     int warp_smem_b_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1354     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1355     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1356     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1357     uint32_t lane_smem_b_ptr =
1358         (smem_b_base_ptr +
1359          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1360           sizeof(half));
1361     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1362 }
1363
1364 // MMA compute: 发射 VAL_TILE_M × VAL_TILE_N 条 MMA 指令
1365 #pragma unroll
1366 for (int i = 0; i < VAL_TILE_M; ++i) {
1367     #pragma unroll
1368     for (int j = 0; j < VAL_TILE_N; ++j) {
1369         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1370                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1371                  RB[j][0], RB[j][1], // B fragment
1372                  RC[i][j][0], RC[i][j][1]);
1373     }
1374 }
1375
1376 // 自适应等待: 流水线满载期用 K_STAGE-2, 尾部排空用 0
1377 if (k + K_STAGE - 1 < NUM_K_TILES) {
1378     CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1379 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1380     CP_ASYNC_WAIT_GROUP(0);
1381 }
1382 __syncthreads();
1383 }
1384
1385 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)

```

```

1386 {
1387     for (int i = 0; i < VAL_TILE_M; ++i) {
1388         uint32_t RC0[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1389         uint32_t RC1[VAL_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1390         // =====
1391         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1392         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1393         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1394         //
1395         //      cols: c0-1    c2-3    c4-5    c6-7
1396         //  r0:  T0{c0,c1}   T1      T2      T3      }
1397         //  r1:   T4        T5      T6      T7      }
1398         //  r2:  T8      →   T9      →   T10     →   T11     } RC[0]
1399         //  ...
1400         //  r7:  T28        T29      T30      T31     }
1401         //  r8:  T0{c2,c3}   T1      T2      T3      }
1402         //  r9:   T4        T5      T6      T7      }
1403         //  r10: T8      →   T9      →   T10     →   T11     } RC[1]
1404         //  ...
1405         //  r15: T28        T29      T30      T31     }
1406         //
1407         // Within a 4-lane group (e.g. T0-T3 for row 0):
1408         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1409         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1410         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1411         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1412         // =====
1413 #pragma unroll
1414         for (int j = 0; j < VAL_TILE_N; ++j) {
1415             RC0[j][0] = RC[i][j][0];
1416             RC1[j][0] = RC[i][j][1];
1417             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1418             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1419             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1420             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1421             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1422             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1423         }
1424         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1425         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1426         //
1427         // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1428         // |-----|-----|-----|-----|
1429         // | 0~3   | 0       | row 0     | row 8     |
1430         // | 4~7   | 1       | row 1     | row 9     |
1431         // | 8~11  | 2       | row 2     | row 10    |
1432         // | 12~15 | 3       | row 3     | row 11    |
1433         // | 16~19 | 4       | row 4     | row 12    |
1434         // | 20~23 | 5       | row 5     | row 13    |
1435         // | 24~27 | 6       | row 6     | row 14    |
1436         // | 28~31 | 7       | row 7     | row 15    |
1437         //
1438         if (lane_id % 4 == 0) {
1439             // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1440             int store_warp_smem_c_m = warp_m * (MMA_M * VAL_TILE_M) + i * MMA_M;
1441             int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1442 #pragma unroll
1443             for (int j = 0; j < VAL_TILE_N; ++j) {
1444                 // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1445                 int store_warp_smem_c_n = warp_n * (MMA_N * VAL_TILE_N) + j * MMA_N;
1446                 int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1447                 int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1448                 int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;

```



```

1449 // 128-bit store: 一次写入 8 个 half
1450 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1451     *reinterpret_cast<float4*>(&RC0[j][0]);
1452 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1453     *reinterpret_cast<float4*>(&RC1[j][0]);
1454 }
1455 }
1456 }
1457 }
1458 }
1459
1460 // =====
1461 // Phase 7c: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1462 // =====
1463 // 面试要点 (WGMMMA vs MMA 对比):
1464 //   - MMA: warp 级 (32 threads), 同步执行
1465 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
1466 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
1467 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1468 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
1469 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1470
1471 // ---- WGMMMA 辅助函数 ----
1472 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7):
1473 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
1474 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
1475 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
1476 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
1477 //     都是 "wait until only N or fewer groups are pending"。
1478 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1479 #define WGMMMA_COMMIT_GROUP() \
1480     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory") \
1481 #define WGMMMA_WAIT_GROUP(n) \
1482     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
1483
1484 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1485 // 编码规则 (PTX ISA §9.7.15.5.1.2.2):
1486 //   encoded = (x & 0x3FFFF) >> 4
1487 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1488 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1489 // 可省略以扩大可表示范围。
1490 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
1491 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1492
1493 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1494 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
1495 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1496 //
1497 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 与 CUTLASS GmmaDescriptor):
1498 // -----
1499 // | 位域          | 范围      | 大小 | 含义
1500 // |-----|-----|-----|-----|
1501 // | start_address  | [0, 14)   | 14   | smem 基址编码
1502 // | (unused)       | [14, 16)  | 2    | -
1503 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
1504 // | (unused)       | [30, 32)  | 2    | -
1505 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
1506 // | (unused)       | [46, 49)  | 3    | -
1507 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
1508 // | (unused)       | [52, 62)  | 10   | -
1509 // | layout_type    | [62, 64)  | 2    | swizzle 模式
1510 // -----
1511 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle

```

```

1512 //
1513 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
1514 //   - 128B swizzle atom 在 128-bit 单位下为 8×8
1515 //   - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
1516 //   - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
1517 //   - 这是 stride_byte_offset=1024 的来源
1518 //
1519 //   - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
1520 //     此处填 16 仅为占位, 编码后为 1。
1521 //
1522 //   - base_offset=0 (bits [49,52] 未显式置位) , 表示 smem ptr 已 1024B 对齐。
1523 //     若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1524 //
1525 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1526 __device__ inline uint64_t make_smem_desc(half *ptr) {
1527     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1528     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
1529     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1530
1531     // 从全零开始: 所有位域初始化为 0。
1532     uint64_t desc = 0x0000000000000000;
1533
1534     // — bits [0, 14): start_address —
1535     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
1536     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1537     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
1538     desc |= SMEM_DESC_ENCODE(addr);
1539
1540     // — bits [16, 30): leading_byte_offset —
1541     // 原始值 16 (字节) , 编码后为 (16 & 0x3FFFF) >> 4 = 1。
1542     // << 16 将编码值放到 bits [16, 30)。
1543     // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
1544     // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1545     // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1546     //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
1547     //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
1548     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1549
1550     // — bits [32, 46): stride_byte_offset —
1551     // 原始值 1024 (字节) , 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
1552     // << 32 将编码值放到 bits [32, 46)。
1553     // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
1554     // 1024 的来源 (K-Major + 128B swizzle + half) :
1555     //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
1556     //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
1557     // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
1558     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1559
1560     // — bits [62, 64): layout_type (swizzle mode) —
1561     // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1562     // 1 = 128B swizzle mode。
1563     // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1564     // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
1565     // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1566     desc |= 1llu << 62;
1567
1568     return desc;
1569 }
1570
1571 // ---- WGMMMA PTX 指令宏 ----
1572 // wmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1573 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算 D[64×128] += A[64×16] × B[16×128]。
1574 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度) 。

```

```

1575 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
1576 // 每线程 64 half = 32 uint32。
1577 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
1578 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
1579 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
1580 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
1581 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
1582 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
1583                                     TransB) \
1584 { \
1585     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1586     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1587     asm volatile( \
1588         "{\n" \
1589         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1590         "%0, %1, %2, %3, %4, %5, %6, %7, " \
1591         "%8, %9, %10, %11, %12, %13, %14, %15, " \
1592         "%16, %17, %18, %19, %20, %21, %22, %23, " \
1593         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
1594         "%32, " \
1595         "%33, " \
1596         "%34, %35, %36, %37, %38;\n" \
1597         "}\n" \
1598         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1599         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1600         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1601         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1602         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1603         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1604         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1605         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1606         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
1607         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1608         "n"(int32_t(TransB))); \
1609 }
1610
1611 // ---- TMA Shared Memory Layout ----
1612 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1613 //
1614 // 每个 stage 包含两块 smem:
1615 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1616 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1617 //
1618 // ★ 关键区别 — WGMMMA 的 B 布局 vs MMA(TN) 的 B^T 布局:
1619 //   MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 解出 col-major B fragment。
1620 //   WGMMMA: smem 存 B [BK, BN] row-major (即 N 维连续, K 维步幅=BN个half)。
1621 //   WGMMMA 指令的 imm-trans-b=0 指示硬件按 K-major 读 B,
1622 //   128B swizzle + make_smem_desc 的 stride_byte_offset 将这些地址
1623 //   重新映射, 使硬件能从 [BK, BN] row-major 中正确按 K-major 顺序取值。
1624 //   简言之: swizzle 桥接了 "row-major 存储" 和 "K-major 读取" 的差异。
1625 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1626     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1627     // B: K-major 布局, 供 WGMMMA imm-trans-b=0 直接读取, [BK, BN]。
1628     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN]
1629 };
1630
1631 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1632 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
1633 //
1634 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1635 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
1636 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1637 //

```

```

1638 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1639 // 将 A/B tile 从 HBM 搬到 shared memory。
1640 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMA 矩阵乘。
1641 //
1642 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1643 // - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
1644 // - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
1645 //
1646 // 本节重点理解:
1647 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1648 // 即可搬运整个 2D tile, 无需所有线程参与。
1649 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1650 // 3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1651 // Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
1652 //
1653 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
1654 // WGMMA Atom: m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1655 // K Tile: BK=64, 每个 K tile 包含 BK/WGMMA_K=4 个 WGMMA atom
1656 // M Tile: BM=128, 每个 M tile 包含 BM/WGMMA_M=2 个 WGMMA atom
1657 // N Tile: BN=128, 单个 WGMMA_N=128 即可覆盖, 无需在 N 方向分块
1658 // Block Tile: C[128,128] = A[128,64] × B[64,128]
1659 // Threads: 256 = 2 warpgroups × 128 threads/warpgroup
1660 // Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
1661 // Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMA 和写回)
1662 //
1663 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
1664 // 的延迟被计算完全掩盖。
1665 //
1666 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1667 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
1668 // Block: (256, 1, 1), 2 warpgroups
1669 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1670 template <const int WGMMA_M = 64, const int WGMMA_N = 128,
1671          const int WGMMA_K = 16, const int BM = 128, const int BN = 128,
1672          const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1673          const bool BLOCK_SWIZZLE = false>
1674 __global__ void __launch_bounds__(NUM_THREADS)
1675 hgemm_wgmma_stages_tn(
1676     int M, int N, int K, half *C,
1677     const CUtensorMap *__restrict__ tensorMapA,
1678     const CUtensorMap *__restrict__ tensorMapB) {
1679     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1680     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
1681     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
1682     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1683     // 不展开宿主侧 create_tensor_map 细节。
1684
1685     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1686     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1687     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1688     const int by = blockIdx.y;
1689     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1690     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1691     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1692     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1693     // 当只有一个 consumer 时, 它负责全部 BM=128 行
1694     constexpr int B_WG_M = BM / num_consumers; // 128
1695
1696     // 边界检查: 确保当前 block 不超出 M/N 范围
1697     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1698         return;
1699
1700     // ---- Shared Memory 分配 ----

```

```

1701 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1702 // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1703 extern __shared__ __align__(128) uint8_t smem_AB[];
1704 WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1705     *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE> *>(smem_AB);
1706 half *s_a = s.A;
1707 half *s_b = s.B;
1708
1709 // ---- cuda::barrier 初始化 ----
1710 //
1711 // ★ Barrier 机制速查 (arrive / wait 核心语义) :
1712 //
1713 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
1714 //
1715 //   init(bar, N): 设置 arrive_count = N。
1716 //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
1717 //
1718 //   arrive(): 线程声明"我已到达此 barrier 点"。
1719 //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
1720 //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
1721 //
1722 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
1723 //       - 即: 等待 phase 翻转。
1724 //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
1725 //                           (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
1726 //
1727 //   典型用法: b.wait(b.arrive())
1728 //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
1729 //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
1730 //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
1731 //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
1732 //
1733 // ★ 本 kernel 的 Pipeline 同步协议 (K_STAGE=3, arrive_count=129) :
1734 //
1735 //   Producer (1 thread)           Consumer (128 threads)
1736 //   -----
1737 //   C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
1738 //   ┌ for each k_tile: ┐           ┌ for each k_tile: ┐
1739 //   │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q]) │
1740 //   │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪   │
1741 //   │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算   │
1742 //   │   ↓ 异步拷贝               │ │ C3: wait WGMMMA 完成 │
1743 //   │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q]) │
1744 //   │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完 │
1745 //   └──────────────────┘         └──────────────────┘
1746 //
1747 //   关键不变式 (invariant) :
1748 //       - Producer 不会覆盖 Consumer 正在读的 stage
1749 //       - Consumer 不会读 Producer 还没写完的 stage
1750 //       - 通过 full/empty 两个 barrier 的 phase 交替来保证
1751 //
1752 //   每个 stage 有两个 barrier:
1753 //       full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
1754 //       empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
1755 //
1756 //   arrive_count = 128 (consumer) + 1 (producer) = 129:
1757 //       每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
1758 #pragma nv_diag_suppress static_var_with_dynamic_init
1759 __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1760 __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1761 #pragma nv_diag_default static_var_with_dynamic_init
1762
1763 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。

```

```

1764 const int num_blocks_k = K / BK;
1765 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1766 const int tid = threadIdx.x % 128;    // 0~127 within warpgroup
1767
1768 // 初始化 barriers (仅 thread 0 执行)
1769 if (threadIdx.x == 0) {
1770     for (int i = 0; i < K_STAGE; ++i) {
1771         // arrive_count = 129: 128 consumer + 1 producer
1772         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
1773         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
1774         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
1775         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1776         init(&empty[i], num_consumers * 128 + 1); // same
1777     }
1778     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1779     // 对 async proxy (TMA/WGMMA) 可见。
1780     cuda::device::experimental::fence_proxy_async_shared_cta();
1781 }
1782 __syncthreads();
1783
1784 // =====
1785 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
1786 // =====
1787 //
1788 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
1789 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
1790 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
1791 //
1792 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
1793 //   Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
1794 //   Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
1795 //
1796 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
1797 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
1798 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
1799 //
1800 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
1801 // 交替天然保证了流水线串行化 (at most K_STAGE-1 steps ahead)。
1802 // =====
1803 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1804 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
1805 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
1806 // =====
1807 if (wg_idx == 0) {
1808     if (tid == 0) {
1809         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1810         int qidx = 0;
1811         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1812              ++block_k_iter, ++qidx) {
1813             if (qidx == K_STAGE)
1814                 qidx = 0;
1815
1816             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
1817             //
1818             // 模式 empty[qidx].wait(empty[qidx].arrive()):
1819             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
1820             //   获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
1821             //   128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
1822             //   → phase 立即翻转。
1823             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
1824             //   由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
1825             //   wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
1826             //

```



```

1827 // 语义: Producer 说"我准备好写 stage qidx 了, Consumer 用完没有? "
1828 //      如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
1829 //      如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
1830 empty[qidx].wait(empty[qidx].arrive());
1831
1832 // Step P2: 提交 TMA 2D 拷贝指令
1833 // cp_async_bulk_tensor_2d_global_to_shared:
1834 //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1835 //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1836 //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1837 //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1838 //
1839 // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
1840 // 无需线程逐元素搬运, 零寄存器开销。
1841 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1842 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1843     &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1844     full[qidx]);
1845
1846 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1847 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1848     &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1849     full[qidx]);
1850
1851 // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
1852 //
1853 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
1854 //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
1855 //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
1856 //   - Phase 翻转条件:
1857 //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
1858 //       (b) 所有声明的 async 字节已写入 smem
1859 //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
1860 //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
1861 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
1862     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1863 }
1864 }
1865 }
1866 // =====
1867 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1868 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1869 // 所有 128 个线程 (4 warps) 全部参与。
1870 // =====
1871 else {
1872     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
1873     //
1874     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
1875     // 这是 Pipeline 的"起搏"步骤——没有它, Producer 的 empty[qidx].wait()
1876     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
1877     //
1878     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
1879     // Producer 后续的 empty[qidx].arrive() 作为第 129 次, 触发 phase 翻转。
1880     for (int i = 0; i < K_STAGE; ++i) {
1881         [[maybe_unused]] auto token = empty[i].arrive();
1882     }
1883
1884     // 累加器寄存器声明
1885     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
1886     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
1887     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
1888     //   - d[*][g][*]: N 方向第 g 组 16 列
1889     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16×16 子块)

```



```

1890 // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
1891 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
1892 uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1893
1894 int qidx = 0;
1895 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
1896 for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1897      ++block_k_iter, ++qidx) {
1898     if (qidx == K_STAGE)
1899         qidx = 0;
1900
1901     // Step C1: 等待 TMA 数据就绪 (full 信号)
1902     //
1903     // 模式 full[qidx].wait(full[qidx].arrive()):
1904     //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
1905     //   - 当前 phase 累积: 128/129, phase 尚未翻转。
1906     //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
1907     //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
1908     //
1909     // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有? "
1910     full[qidx].wait(full[qidx].arrive());
1911
1912     // Step C2: 发射 WGMMMA 指令序列
1913     //
1914     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
1915     // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
1916     //
1917     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
1918     //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
1919     //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
1920     //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
1921     //
1922     // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
1923     //   D[64×128] += A[64×16] × B[16×128]
1924     //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
1925     //   ScaleD=1: 累加。K 维有 BK/WGMMMA_K=4 次迭代, 每次都 ScaleD=1。
1926     WGMMMA_FENCE();
1927
1928     // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
1929     // 每个 atom 处理 64 行 M 方向数据。
1930 #pragma unroll
1931     for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1932         // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
1933         // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
1934         half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1935
1936         // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
1937         // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
1938 #pragma unroll
1939         for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1940             // 第 k_it 次 WGMMMA:
1941             //   A: wgmma_sA + k_it * WGMMMA_K (A 的 K 维起始位置)
1942             //   B: s_b + qidx * BK * BN + k_it * WGMMMA_K (B 的 K 维起始位置)
1943             //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
1944             //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
1945             //   ScaleD=1: 累加 (K 维迭代需要累积结果)
1946             WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
1947                                           s_b + qidx * BK * BN + k_it * WGMMMA_K,
1948                                           1, // ScaleD=1: accumulate (不清零)
1949                                           1, 1, 0, 0);
1950         }
1951     }
1952 }

```

```

1953 // Step C3: 提交并等待 WGMMMA 完成
1954 //
1955 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
1956 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
1957 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
1958 //
1959 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
1960 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
1961 // * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
1962 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
1963 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
1964 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
1965 WGMMMA_COMMIT_GROUP();
1966 WGMMMA_WAIT_GROUP(0);
1967
1968 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
1969 //
1970 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
1971 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
1972 //
1973 // 语义: Consumer 说"stage qidx 我已经读完了, 你可以放心覆盖了。"
1974 [[maybe_unused]] auto token = empty[qidx].arrive();
1975 }
1976
1977 // =====
1978 // Epilogue: 将寄存器中的累加结果写回 global memory C
1979 // =====
1980 //
1981 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
1982 //
1983 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
1984 // 这些 half 分布在 d[2][8][4] 数组中:
1985 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
1986 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
1987 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
1988 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
1989 //
1990 // 其中:
1991 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
1992 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
1993 //
1994 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
1995 // +-----+-----+
1996 // | reg[0] | reg[2] | <- rows [row, row+8)
1997 // |(col,+2)|(col+8)|
1998 // +-----+-----+
1999 // | reg[1] | reg[3] | <- rows [row+8, row+16)
2000 // |(col,+2)|(col+8)|
2001 // +-----+-----+
2002 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
2003 //
2004 // 三维遍历:
2005 // m_it=0: rows 0~63, m_it=1: rows 64~127
2006 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
2007 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
2008 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
2009
2010 const int lane = tid % 32;
2011 const int warp = tid / 32;
2012 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
2013 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
2014 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
2015 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。

```

```

2018 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
2019 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0) 。
2020 const int row = warp * 16 + lane / 4;
2021 // block_C: 当前 C tile 在 global memory 中的起始地址
2022 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
2023 half *block_C = C + by * BM * N + bx * BN;
2024
2025 #pragma unroll
2026 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
2027     int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
2028     #pragma unroll
2029     for (int g = 0; g < WGMMMA_N / 16; ++g) {
2030         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
2031         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
2032         // 左上象限: (row, col)
2033         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
2034             d[m_it][g][0];
2035         // 左下象限: (row+8, col)
2036         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
2037             d[m_it][g][1];
2038         // 右上象限: (row, col+8)
2039         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
2040             d[m_it][g][2];
2041         // 右下象限: (row+8, col+8)
2042         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
2043             d[m_it][g][3];
2044     }
2045 }
2046
2047 #if (defined(NOTES_V2_HAS_WGMMMA) \
2048     || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
2049     || defined(__CUDA_ARCH_FEAT_SM90_ALL))
2050 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
2051 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
2052 // - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
2053 //   以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
2054 // - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
2055 //   对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
2056 // - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
2057 // - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
2058 // - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
2059 //
2060 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
2061
2062 template <int BlockMajorSize, int BlockMinorSize>
2063 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
2064         half *gmem_ptr,
2065         int blocks_height,
2066         int blocks_width) {
2067     void *gmem_address = (void *)gmem_ptr;
2068     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
2069                                     (uint64_t)BlockMajorSize * blocks_height,
2070                                     1, 1, 1};
2071     uint64_t gmem_prob_stride[5] = {
2072         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2073     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2074                                    uint32_t(BlockMajorSize), 1, 1, 1};
2075     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2076     CUresult result = cuTensorMapEncodeTiled(
2077         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2078         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,

```

```

2079     CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2080     CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
2081     if (result != CUDA_SUCCESS)
2082         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2083 }
2084
2085 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2086     half *src, int blocks_height, int blocks_width) {
2087     CUTensorMap *tma_map_d;
2088     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2089     CUTensorMap tma_map_host;
2090     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2091     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2092         cudaMemcpyHostToDevice);
2093     return tma_map_d;
2094 }
2095 #endif /* NOTES_V2_HAS_WGMMMA */
2096
2097 // =====
2098 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2099 // =====
2100 // 面试要点 (FlashAttention 算法) :
2101 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2102 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2103 // 2. FA 三板斧:
2104 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
2105 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2106 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2107 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2108 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2109 //    - 优点: 减少 warp 间通信和 shuffle
2110 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2111 //    for each K,V tile:
2112 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
2113 //        m_new = max(m_old, row_max(S_cur * scale))
2114 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
2115 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2116 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2117 //        O_final = O_new / l_final
2118 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2119 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
2120 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2121 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
2122 //    - 这是此实现的寄存器布局复用技巧, 不要背成 “所有 MMA A/C fragment
2123 //      都天然同构” 的通用结论
2124 //
2125 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
2126 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
2127 // Grid: ((QKV_seqLen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
2128 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
2129 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
2130
2131 // ---- 寄存器填充辅助函数 ----
2132 template <typename T, int M, const int N, const int K = 2>
2133 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
2134 #pragma unroll
2135     for (int i = 0; i < M; ++i)
2136 #pragma unroll
2137         for (int j = 0; j < N; ++j)
2138 #pragma unroll
2139             for (int k = 0; k < K; ++k)
2140                 R[i][j][k] = val;
2141 }

```

```

2142
2143 template <typename T, int M, const int N = 2>
2144 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2145     #pragma unroll
2146     for (int i = 0; i < M; ++i)
2147     #pragma unroll
2148         for (int j = 0; j < N; ++j)
2149             R[i][j] = val;
2150 }
2151
2152 // =====
2153 // FlashAttention-2 Split-Q Kernel (完整实现)
2154 // =====
2155 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2156 //
2157 // Tile 设计 (以 kHeadDim=64 为例) :
2158 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
2159 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
2160 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2161 //
2162 // 执行流程:
2163 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2164 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2165 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2166 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
2167 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2168 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2169 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2170 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2171 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2172 //   8) 最终 rescale: O_final = (1/l_final) * O_final
2173 //   9) Epilogue: warp shuffle + 128-bit collective store
2174
2175 template <
2176     const int kHeadDim,           // head dim: 32, 64, 128
2177     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
2178     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
2179     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
2180     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
2181     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
2182     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
2183     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
2184     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
2185     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
2186     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
2187     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
2188     const int kStage,             // pipeline stages for K: 1 or 2
2189     const int kPad>              // padding for bank conflict avoidance
2190 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
2191     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2192                                     int QKV_seqlen, int QKV_head) {
2193     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
2194     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
2195     constexpr int kNumThreads = WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2196     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2197     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2198     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2199     const float scale = 1.0f / sqrtf((float)kHeadDim);
2200
2201     // Block indexing
2202     const int QKV_batch_id = blockIdx.y / QKV_head;
2203     const int QKV_head_id = blockIdx.y % QKV_head;
2204     const int Q_tile_id = blockIdx.x;

```

```

2205 const int tid = threadIdx.x;
2206 const int warp_id = tid / WARP_SIZE;
2207 const int lane_id = tid % WARP_SIZE;
2208 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2209 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2210
2211 // Global memory base offsets for this (batch, head)
2212 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2213 const int Q_gmem_offset =
2214     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2215     kHeadDim;
2216 const int K_gmem_offset = Q_gmem_offset;
2217 const int V_gmem_offset = Q_gmem_offset;
2218 const int O_gmem_offset = Q_gmem_offset;
2219
2220 // Thread-to-smem mapping for cooperative load
2221 int load_smem_Q_Br = tid / (kNumThreads / Br);
2222 int load_smem_Q_d =
2223     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2224 int load_smem_K_Bc = tid / (kNumThreads / Bc);
2225 int load_smem_K_d =
2226     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2227 int load_smem_V_Bc = tid / (kNumThreads / Bc);
2228 int load_smem_V_d =
2229     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2230
2231 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2232 if (load_gmem_Q_Br >= QKV_seqlen)
2233     return;
2234
2235 // ---- Shared memory layout ----
2236 extern __shared__ half smem[];
2237 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2238 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2239 half *Q_tile_smem = smem;
2240 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2241 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2242 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2243 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2244
2245 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2246 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2247 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2248
2249 // ---- Online Softmax persistent state ----
2250 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2251 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2252 float lane_block_row_max_old[kValTileSeqLenQ][2];
2253 float lane_block_row_sum_old[kValTileSeqLenQ][2];
2254 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2255 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2256
2257 // ---- Register allocation ----
2258 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
2259 uint32_t R_K[kValTileSeqLenK][2]; // K regs
2260 uint32_t R_V[kValTileHeadDimV][2]; // V regs
2261 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2262 // 对单个 m16n8k16 tile 而言:
2263 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
2264 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
2265 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2266 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
2267 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile

```



```

2268 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
2269         [2]; // 0 accumulator (final output)
2270
2271 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2272 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
2273
2274 // =====
2275 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2276 // =====
2277 {
2278     int load_gmem_Q_addr =
2279         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2280     uint32_t load_smem_Q_ptr =
2281         smem_Q_base_ptr +
2282         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2283 #pragma unroll
2284     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2285         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2286     }
2287     CP_ASYNC_COMMIT_GROUP();
2288 }
2289
2290 // =====
2291 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2292 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
2293 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
2294 // =====
2295 if constexpr (kStage > 1) {
2296 #pragma unroll
2297     for (int stage = 0; stage < (kStage - 1); ++stage) {
2298         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2299         int load_gmem_K_addr =
2300             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2301         uint32_t load_smem_K_ptr =
2302             smem_K_base_ptr +
2303             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2304              load_smem_K_d) *
2305             sizeof(half);
2306 #pragma unroll
2307         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2308             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2309         }
2310         CP_ASYNC_COMMIT_GROUP();
2311     }
2312     CP_ASYNC_WAIT_GROUP(kStage - 2);
2313     __syncthreads();
2314 }
2315
2316 // =====
2317 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2318 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2319 // =====
2320 #pragma unroll 1
2321 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2322     int smem_sel = tile_K_seqlen % kStage;
2323     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2324
2325     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2326     if constexpr (kStage > 1) {
2327         // 只有 kStage>1 才能真正做 K 的 pipeline:
2328         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
2329         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
2330         // Load current V tile (no pipeline for V — one stage is enough)

```



```

2331 {
2332     int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2333     int load_gmem_V_addr =
2334         V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2335     uint32_t load_smem_V_ptr =
2336         smem_V_base_ptr +
2337         (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2338 #pragma unroll
2339     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2340         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2341     }
2342     CP_ASYNC_COMMIT_GROUP();
2343 }
2344
2345 // Prefetch next K tile (pipelined)
2346 if ((tile_K_seqlen + 1) < Tc) {
2347     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2348     int load_gmem_K_addr =
2349         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2350     uint32_t load_smem_K_ptr =
2351         smem_K_base_ptr +
2352         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2353          load_smem_K_d) *
2354         sizeof(half);
2355 #pragma unroll
2356     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2357         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2358     }
2359     CP_ASYNC_COMMIT_GROUP();
2360 }
2361 }
2362
2363 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2364 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2365 #pragma unroll
2366 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2367     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2368 #pragma unroll
2369     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2370         int warp_smem_Q_Br =
2371             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
2372         int lane_smem_Q_Br =
2373             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2374         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2375         uint32_t lane_smem_Q_ptr =
2376             smem_Q_base_ptr +
2377             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2378         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2379                    lane_smem_Q_ptr);
2380     }
2381
2382     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2383     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2384 #pragma unroll
2385     for (int j = 0; j < kValTileSeqLenK; ++j) {
2386         int warp_smem_K_Bc =
2387             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
2388         int lane_smem_K_Bc =
2389             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2390         int lane_smem_K_d =
2391             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2392         uint32_t lane_smem_K_ptr =
2393             smem_K_base_ptr +

```

```

2394         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2395         lane_smem_K_d) *
2396         sizeof(half);
2397     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2398 }
2399
2400 // MMA: S[tile] += Q[tile] @ K^T[tile]
2401 #pragma unroll
2402 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2403 #pragma unroll
2404     for (int j = 0; j < kValTileSeqLenK; ++j) {
2405         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2406                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2407                 R_S[i][j][1]);
2408     }
2409 }
2410 } // end loop over d
2411 __syncthreads();
2412
2413 // =====
2414 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
2415 // =====
2416 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2417 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2418 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2419 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2420 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2421 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2422 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2423 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
2424 // kValTileSeqLenK tiles.
2425
2426 float lane_row_max_new[kValTileSeqLenQ][2];
2427 float lane_row_sum_new[kValTileSeqLenQ][2];
2428 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2429 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2430
2431 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
2432 #pragma unroll
2433 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2434 #pragma unroll
2435     for (int j = 0; j < kValTileSeqLenK; ++j) {
2436         // Extract half values from R_S registers (C matrix fragment layout)
2437         float2 t_reg_S_0 =
2438             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2439         float2 t_reg_S_1 =
2440             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2441         // S = (Q@K^T) * scale
2442         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2443         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2444         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2445         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2446     }
2447     // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2448     // {0,4,8,...,28} hold valid data)
2449     lane_row_max_new[i][0] =
2450         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2451     lane_row_max_new[i][1] =
2452         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2453 }
2454
2455 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
2456 // 面试关键点: 这里将 P 写回 R_S 寄存器!

```

```

2457 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2458 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2459 #pragma unroll
2460 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2461     // m_new = max(m_old, m_cur)
2462     float block_row_max_new_0 =
2463         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2464     float block_row_max_new_1 =
2465         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2466
2467 #pragma unroll
2468 for (int j = 0; j < kValTileSeqLenK; ++j) {
2469     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2470     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2471
2472     // P = exp(S * scale - m_new), 用 fma 保证精度
2473     t_reg_S_0.x =
2474         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2475     t_reg_S_0.y =
2476         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2477     t_reg_S_1.x =
2478         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2479     t_reg_S_1.y =
2480         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2481
2482     // Accumulate row sums
2483     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2484     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2485
2486     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2487     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2488     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2489 }
2490
2491 // Warp-level reduce sum (warp_size = 4, same as max)
2492 lane_row_sum_new[i][0] =
2493     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2494 lane_row_sum_new[i][1] =
2495     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2496 }
2497 __syncthreads();
2498
2499 // =====
2500 // 3d: P@V - P[Br,Bc] @ V[Bc,d] = 0[Br,d]
2501 // =====
2502 // Wait for V to be ready before computing P@V
2503 if constexpr (kStage > 1) {
2504     if ((tile_K_seqLen + 1) < Tc) {
2505         CP_ASYNC_WAIT_GROUP(1);
2506     } else {
2507         CP_ASYNC_WAIT_GROUP(0);
2508     }
2509 } else {
2510     CP_ASYNC_WAIT_GROUP(0);
2511 }
2512 __syncthreads();
2513
2514 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
2515
2516 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2517 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2518 #pragma unroll
2519 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {

```

```

2520 // ldmatrix.x2.trans: load V[Bc,d] with transposition
2521 // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2522 // transposed ldmatrix
2523 #pragma unroll
2524 for (int j = 0; j < kValTileHeadDimV; ++j) {
2525     int warp_smem_V_d =
2526         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2527     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2528     int lane_smem_V_d = warp_smem_V_d;
2529     uint32_t lane_smem_V_ptr =
2530         smem_V_base_ptr +
2531         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2532     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2533 }
2534
2535 // P matrix layout in R_S[i][j][2]:
2536 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2537 //   | 64x64 | warp_KV 0 |
2538 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2539 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2540 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2541 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2542 // tile_V_Bc selects which 16-column slice of P to use:
2543 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2544 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2545 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2546 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2547 // 对应的 MMA A fragment 布局可以这样记:
2548 //   - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
2549 //   - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
2550 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2551 // 才能把 R_S 中的 P 直接喂给 HMMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2552 // fragment 都无条件成立的通用结论。layout转换逻辑:
2553 //   C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
2554     int w = tile_V_Bc * 2;
2555 #pragma unroll
2556     for (int i = 0; i < kValTileSeqLenP; ++i) {
2557 #pragma unroll
2558         for (int j = 0; j < kValTileHeadDimV; ++j) {
2559             HMMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2560                 R_S[i][w][0], R_S[i][w][1], R_S[i][w+1][0], R_S[i][w+1][1], // A fragment = P
2561                 R_V[j][0], R_V[j][1], // B fragment = V
2562                 R_0[i][j][0], R_0[i][j][1]); // C fragment output
2563         }
2564     }
2565 } // end for tile_V_Bc
2566 __syncthreads();
2567
2568 // =====
2569 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
2570 // =====
2571 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
2572 #pragma unroll
2573 for (int i = 0; i < kValTileSeqLenP; ++i) {
2574     float block_row_max_new_0 = lane_row_max_new[i][0];
2575     float block_row_max_new_1 = lane_row_max_new[i][1];
2576     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2577     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2578
2579     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2580     float block_row_max_old_1 = lane_block_row_max_old[i][1];
2581
2582     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);

```

```

2583     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2584
2585     // Handle first iteration: m_old = -inf, need to use m_new directly
2586     block_row_max_old_0 =
2587         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2588     block_row_max_old_1 =
2589         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2590
2591     float rescale_o_factor_0 =
2592         __expf(block_row_max_old_0 - block_row_max_new_0);
2593     float rescale_o_factor_1 =
2594         __expf(block_row_max_old_1 - block_row_max_new_1);
2595
2596     // Rescale O_old + Add P@V in one fused step
2597     #pragma unroll
2598     for (int j = 0; j < kValTileHeadDimV; ++j) {
2599         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
2600         //   reg[0] -> rows 0~7 的 {c0,c1}
2601         //   reg[1] -> rows 8~15 的 {c2,c3}
2602         float2 t_reg_0_0 = __half2float2(HALF2(R_0[i][j][0]));
2603         float2 t_reg_0_1 = __half2float2(HALF2(R_0[i][j][1]));
2604         float2 t_reg_D_0 = __half2float2(HALF2(R_D[i][j][0]));
2605         float2 t_reg_D_1 = __half2float2(HALF2(R_D[i][j][1]));
2606
2607         // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
2608         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2609         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2610         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2611         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2612
2613         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2614         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2615     }
2616
2617     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
2618     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2619     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2620     lane_block_row_sum_old[i][0] = __fmaf_rn(
2621         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2622     lane_block_row_sum_old[i][1] = __fmaf_rn(
2623         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2624
2625     // Update m
2626     lane_block_row_max_old[i][0] = block_row_max_new_0;
2627     lane_block_row_max_old[i][1] = block_row_max_new_1;
2628 }
2629
2630 // Wait for next K tile to be ready in smem before next iteration
2631 if constexpr (kStage > 1) {
2632     if ((tile_K_seqlen + 1) < Tc) {
2633         CP_ASYNC_WAIT_GROUP(0);
2634     }
2635     __syncthreads();
2636 }
2637 } // end outer loop over K seqlen
2638 __syncthreads();
2639
2640 // =====
2641 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
2642 // =====
2643 #pragma unroll
2644 for (int i = 0; i < kValTileSeqLenP; ++i) {
2645     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);

```

```

2646     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2647 #pragma unroll
2648     for (int j = 0; j < kValTileHeadDimV; ++j) {
2649         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2650         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2651         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2652         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2653         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2654         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2655         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2656         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2657     }
2658 }
2659
2660 // =====
2661 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2662 // =====
2663 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2664 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2665 #pragma unroll
2666     for (int i = 0; i < kValTileSeqLenP; ++i) {
2667 #pragma unroll
2668         for (int j = 0; j < kValTileHeadDimV; ++j) {
2669             uint32_t R_Z[2][4];
2670             R_Z[0][0] = R_D[i][j][0];
2671             R_Z[1][0] = R_D[i][j][1];
2672             R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2673             R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2674             R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2675             R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2676             R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2677             R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2678
2679             // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2680             if (lane_id % 4 == 0) {
2681                 int store_warp_regs_0_Br =
2682                     warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
2683                 int store_lane_gmem_0_Br =
2684                     Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2685                 int store_warp_regs_0_d =
2686                     warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2687                 int store_lane_gmem_0_d = store_warp_regs_0_d;
2688                 int store_gmem_0_addr_0 = 0_gmem_offset +
2689                     (store_lane_gmem_0_Br + 0) * kHeadDim +
2690                     store_lane_gmem_0_d;
2691                 int store_gmem_0_addr_1 = 0_gmem_offset +
2692                     (store_lane_gmem_0_Br + 8) * kHeadDim +
2693                     store_lane_gmem_0_d;
2694                 // LDST128BITS = reinterpret_cast<float4*>
2695                 *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2696                     *reinterpret_cast<float4*>(&R_Z[0][0]);
2697                 *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2698                     *reinterpret_cast<float4*>(&R_Z[1][0]);
2699             }
2700         }
2701     }
2702 }
2703
2704 // =====
2705 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
2706 // =====
2707
2708 static inline void check(cudaError_t err, const char *msg) {

```

```

2709     if (err != cudaSuccess) {
2710         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2711         exit(EXIT_FAILURE);
2712     }
2713 }
2714
2715
2716 static void test_block_reduce(int N) {
2717
2718     srand(42);
2719     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2720     for (int i = 0; i < N; i++)
2721         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2722
2723     // CPU reference: sum of all elements
2724     double ref = 0.0;
2725     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2726
2727     float *d_a, *d_y;
2728     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2729     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2730
2731     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2732     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2733
2734     dim3 block(128);
2735     dim3 grid((N + 127) / 128);
2736     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
2737     check(cudaGetLastError(), "blockreduce launch");
2738     check(cudaDeviceSynchronize(), "blockreduce sync");
2739
2740     float result;
2741     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2742
2743     float err = fabsf(result - (float)ref);
2744     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
2745         err < 1e-2f ? "PASS" : "FAIL");
2746
2747     free(h_a);
2748     cudaFree(d_a); cudaFree(d_y);
2749 }
2750
2751
2752 static void test_dot(int N) {
2753
2754     srand(42);
2755     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2756     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2757     for (int i = 0; i < N; i++) {
2758         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2759         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2760     }
2761
2762     // CPU reference
2763     double ref = 0.0;
2764     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2765
2766     float *d_a, *d_b, *d_y;
2767     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2768     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2769     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
2770
2771     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");

```



```

2772 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2773 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2774
2775 dim3 block(128);
2776 dim3 grid((N + 127) / 128);
2777 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
2778 check(cudaGetLastError(), "dot launch");
2779 check(cudaDeviceSynchronize(), "dot sync");
2780
2781 float result;
2782 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2783
2784 float err = fabsf(result - (float)ref);
2785 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2786     err < 1e-2f ? "PASS" : "FAIL");
2787
2788 // ---- Dot Vec4 ----
2789 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2790 dim3 block_v4(32);
2791 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
2792 check(cudaGetLastError(), "dot_vec4 launch");
2793 check(cudaDeviceSynchronize(), "dot_vec4 sync");
2794
2795 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2796 float err_v4 = fabsf(result - (float)ref);
2797 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2798
2799 free(h_a); free(h_b);
2800 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2801 }
2802
2803
2804 static void test_relu(int N) {
2805     srand(42);
2806     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2807     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2808     for (int i = 0; i < N; i++)
2809         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2810
2811     float *d_x, *d_y;
2812     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2813     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2814     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2815
2816     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
2817     dim3 block256(256);
2818     dim3 grid256((N + 255) / 256);
2819     relu<<<grid256, block256>>>(d_x, d_y, N);
2820     check(cudaGetLastError(), "relu launch");
2821     check(cudaDeviceSynchronize(), "relu sync");
2822     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2823     float max_err = 0.0f;
2824     for (int i = 0; i < N; i++) {
2825         float expected = fmaxf(0.0f, h_x[i]);
2826         float err = fabsf(h_y[i] - expected);
2827         if (err > max_err) max_err = err;
2828     }
2829     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2830
2831     dim3 block64(64);
2832     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
2833     check(cudaGetLastError(), "relu_vec4 launch");
2834     check(cudaDeviceSynchronize(), "relu_vec4 sync");

```

```

2835 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2836 max_err = 0.0f;
2837 for (int i = 0; i < N; i++) {
2838     float expected = fmaxf(0.0f, h_x[i]);
2839     float err = fabsf(h_y[i] - expected);
2840     if (err > max_err) max_err = err;
2841 }
2842 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2843
2844 free(h_x); free(h_y);
2845 cudaFree(d_x); cudaFree(d_y);
2846 }
2847
2848
2849 static void test_elementwise(int N) {
2850     srand(42);
2851     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2852     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2853     for (int i = 0; i < N; i++) {
2854         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2855         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2856     }
2857
2858     float *d_a, *d_b, *d_c;
2859     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2860     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2861     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2862     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2863     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
2864
2865     dim3 block256(256);
2866     dim3 grid256((N + 255) / 256);
2867     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2868     check(cudaGetLastError(), "eadd launch");
2869     check(cudaDeviceSynchronize(), "eadd sync");
2870     float *h_c = (float *)malloc((size_t)N * sizeof(float));
2871     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2872     float max_err = 0.0f;
2873     for (int i = 0; i < N; i++) {
2874         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2875         if (err > max_err) max_err = err;
2876     }
2877     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2878
2879     dim3 block64(64);
2880     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
2881     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
2882     check(cudaGetLastError(), "eadd_vec4 launch");
2883     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
2884     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
2885     max_err = 0.0f;
2886     for (int i = 0; i < N; i++) {
2887         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2888         if (err > max_err) max_err = err;
2889     }
2890     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2891
2892     free(h_a); free(h_b); free(h_c);
2893     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
2894 }
2895
2896
2897 static void test_histogram(int N) {

```

```

2898 srand(42);
2899 int BINS = 16;
2900 int *h_hist = (int *)calloc(BINS, sizeof(int));
2901 int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
2902 int *h_idx = (int *)malloc((size_t)N * sizeof(int));
2903 for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
2904 for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
2905
2906 int *d_idx, *d_hist;
2907 check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
2908 check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
2909 check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
2910 check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
2911
2912 dim3 block256(256);
2913 dim3 grid256((N + 255) / 256);
2914 histogram<<<grid256, block256>>>(d_idx, d_hist, N);
2915 check(cudaGetLastError(), "histogram launch");
2916 check(cudaDeviceSynchronize(), "histogram sync");
2917 check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
2918
2919 float max_err = 0.0f;
2920 for (int i = 0; i < BINS; i++) {
2921     float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
2922     if (err > max_err) max_err = err;
2923 }
2924 printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2925
2926 free(h_hist); free(h_hist_ref); free(h_idx);
2927 cudaFree(d_idx); cudaFree(d_hist);
2928 }
2929
2930
2931 static void test_softmax(int N) {
2932     // Use N = blockDim.x so one block processes all elements as one token.
2933     constexpr int NUM_THREADS = 256;
2934     if (N != NUM_THREADS) {
2935         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
2936         return;
2937     }
2938
2939     srand(42);
2940     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2941     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
2942     for (int i = 0; i < N; i++)
2943         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
2944
2945     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
2946     float max_val = -FLT_MAX;
2947     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
2948     double sum_exp = 0.0;
2949     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
2950     for (int i = 0; i < N; i++)
2951         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
2952
2953     float *d_x, *d_y;
2954     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
2955     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
2956
2957     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
2958
2959     dim3 block(256);
2960     dim3 grid(1); // one token covering all N elements

```

```

2961 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2962 check(cudaGetLastError(), "softmax launch");
2963 check(cudaDeviceSynchronize(), "softmax sync");
2964
2965 float *h_y = (float *)malloc((size_t)N * sizeof(float));
2966 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
2967
2968 float max_err = 0.0f;
2969 for (int i = 0; i < N; i++) {
2970     float err = fabsf(h_y[i] - h_y_ref[i]);
2971     if (err > max_err) max_err = err;
2972 }
2973 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2974
2975 // ---- Safe Softmax ----
2976 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2977 check(cudaGetLastError(), "safe_softmax launch");
2978 check(cudaDeviceSynchronize(), "safe_softmax sync");
2979 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
2980 max_err = 0.0f;
2981 for (int i = 0; i < N; i++) {
2982     float err = fabsf(h_y[i] - h_y_ref[i]);
2983     if (err > max_err) max_err = err;
2984 }
2985 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2986
2987 // ---- Naive Softmax ----
2988 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2989 check(cudaGetLastError(), "naive_softmax launch");
2990 check(cudaDeviceSynchronize(), "naive_softmax sync");
2991 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
2992 max_err = 0.0f;
2993 for (int i = 0; i < N; i++) {
2994     float err = fabsf(h_y[i] - h_y_ref[i]);
2995     if (err > max_err) max_err = err;
2996 }
2997 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2998
2999 free(h_x); free(h_y); free(h_y_ref);
3000 cudaFree(d_x); cudaFree(d_y);
3001 }
3002
3003
3004 static void test_rms_norm(int N, int K) {
3005
3006     srand(42);
3007     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
3008     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3009     for (int i = 0; i < N * K; i++)
3010         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3011     float g = 1.5f; // gain
3012
3013     // CPU reference: y = (x / rms(x)) * g
3014     float epsilon = 1e-5f;
3015     for (int n = 0; n < N; n++) {
3016         double sum_sq = 0.0;
3017         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
3018         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
3019         for (int k = 0; k < K; k++)
3020             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
3021     }
3022
3023     float *d_x, *d_y;

```

```

3024 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
3025 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
3026 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
3027
3028 dim3 block(128);
3029 dim3 grid(N);
3030 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
3031 check(cudaGetLastError(), "rmsnorm launch");
3032 check(cudaDeviceSynchronize(), "rmsnorm sync");
3033
3034 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3035 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
3036
3037 float max_err = 0.0f;
3038 for (int i = 0; i < N * K; i++) {
3039     float err = fabsf(h_y[i] - h_y_ref[i]);
3040     if (err > max_err) max_err = err;
3041 }
3042 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3043
3044 // ---- RMS Norm Vec4 ----
3045 dim3 block_rv4(32);
3046 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
3047 check(cudaGetLastError(), "rmsnorm_vec4 launch");
3048 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
3049 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
3050 max_err = 0.0f;
3051 for (int i = 0; i < N * K; i++) {
3052     float err = fabsf(h_y[i] - h_y_ref[i]);
3053     if (err > max_err) max_err = err;
3054 }
3055 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3056
3057 free(h_x); free(h_y); free(h_y_ref);
3058 cudaFree(d_x); cudaFree(d_y);
3059 }
3060
3061
3062 static void test_layer_norm(int N, int K) {
3063
3064     srand(42);
3065     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
3066     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3067     for (int i = 0; i < N * K; i++)
3068         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3069     float g = 1.5f, b = 0.3f; // gain and bias
3070
3071     // CPU reference: y = ((x - mean) / std) * g + b
3072     float epsilon = 1e-5f;
3073     for (int n = 0; n < N; n++) {
3074         double sum = 0.0;
3075         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
3076         float mean = (float)sum / (float)K;
3077         double sum_sq = 0.0;
3078         for (int k = 0; k < K; k++) {
3079             float diff = h_x[n * K + k] - mean;
3080             sum_sq += (double)diff * (double)diff;
3081         }
3082         float std = sqrtf((float)sum_sq / (float)K + epsilon);
3083         for (int k = 0; k < K; k++)
3084             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
3085     }
3086

```

```

3087 float *d_x, *d_y;
3088 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
3089 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
3090 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
3091
3092 dim3 block(128);
3093 dim3 grid(N);
3094 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
3095 check(cudaGetLastError(), "layernorm launch");
3096 check(cudaDeviceSynchronize(), "layernorm sync");
3097
3098 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3099 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
3100
3101 float max_err = 0.0f;
3102 for (int i = 0; i < N * K; i++) {
3103     float err = fabsf(h_y[i] - h_y_ref[i]);
3104     if (err > max_err) max_err = err;
3105 }
3106 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3107
3108 // ---- Layer Norm Vec4 ----
3109 dim3 block_lv4(32);
3110 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
3111 check(cudaGetLastError(), "layernorm_vec4 launch");
3112 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
3113 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
3114 max_err = 0.0f;
3115 for (int i = 0; i < N * K; i++) {
3116     float err = fabsf(h_y[i] - h_y_ref[i]);
3117     if (err > max_err) max_err = err;
3118 }
3119 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3120
3121 free(h_x); free(h_y); free(h_y_ref);
3122 cudaFree(d_x); cudaFree(d_y);
3123 }
3124
3125
3126 static void test_rope(int seq_len, int N) {
3127
3128     int total_pairs = seq_len * N;
3129     int total_elems = total_pairs * 2;
3130     size_t size = (size_t)total_elems * sizeof(float);
3131
3132     float *h_x = (float *)malloc(size);
3133     float *h_y_ref = (float *)malloc(size);
3134
3135     srand(42);
3136     for (int i = 0; i < total_elems; i++)
3137         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3138
3139     // CPU reference: 2D rotation for each pair
3140     for (int idx = 0; idx < total_pairs; idx++) {
3141         int token_pos = idx / N;
3142         int token_idx = idx % N;
3143         float x1 = h_x[idx * 2];
3144         float x2 = h_x[idx * 2 + 1];
3145         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3146         float angle = (float)token_pos * theta;
3147         float cos_v = cosf(angle);
3148         float sin_v = sinf(angle);
3149         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;

```

```

3150     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3151 }
3152
3153 float *d_x, *d_y;
3154 check(cudaMalloc(&d_x, size), "rope alloc X");
3155 check(cudaMalloc(&d_y, size), "rope alloc Y");
3156 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3157
3158 dim3 block(256);
3159 dim3 grid((total_pairs + 255) / 256);
3160 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3161 check(cudaGetLastError(), "rope launch");
3162 check(cudaDeviceSynchronize(), "rope sync");
3163
3164 float *h_y = (float *)malloc(size);
3165 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3166
3167 float max_err = 0.0f;
3168 for (int i = 0; i < total_elems; i++) {
3169     float err = fabsf(h_y[i] - h_y_ref[i]);
3170     if (err > max_err) max_err = err;
3171 }
3172 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3173
3174 free(h_x);
3175 free(h_y);
3176 free(h_y_ref);
3177 cudaFree(d_x);
3178 cudaFree(d_y);
3179 }
3180
3181 static void test_mat_transpose(int row, int col) {
3182
3183     size_t size_in = (size_t)row * col * sizeof(float);
3184     size_t size_out = (size_t)col * row * sizeof(float);
3185
3186     float *h_x = (float *)malloc(size_in);
3187     float *h_y_ref = (float *)malloc(size_out);
3188
3189     srand(42);
3190     for (int i = 0; i < row * col; i++)
3191         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3192
3193     // CPU reference: y[j][i] = x[i][j] (row-major)
3194     for (int i = 0; i < row; i++)
3195         for (int j = 0; j < col; j++)
3196             h_y_ref[j * row + i] = h_x[i * col + j];
3197
3198     float *d_x, *d_y;
3199     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3200     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3201     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3202
3203     dim3 block(16, 16);
3204     dim3 grid((col + 15) / 16, (row + 15) / 16);
3205     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3206     check(cudaGetLastError(), "mattrans launch");
3207     check(cudaDeviceSynchronize(), "mattrans sync");
3208
3209     float *h_y = (float *)malloc(size_out);
3210     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
3211
3212

```



```

3213 float max_err = 0.0f;
3214 for (int i = 0; i < col * row; i++) {
3215     float err = fabsf(h_y[i] - h_y_ref[i]);
3216     if (err > max_err) max_err = err;
3217 }
3218 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3219
3220 free(h_x);
3221 free(h_y);
3222 free(h_y_ref);
3223 cudaFree(d_x);
3224 cudaFree(d_y);
3225 }
3226
3227
3228 static void test_mat_transpose_padded(int row, int col) {
3229
3230     size_t size_in = (size_t)row * col * sizeof(float);
3231     size_t size_out = (size_t)col * row * sizeof(float);
3232
3233     float *h_x = (float *)malloc(size_in);
3234     float *h_y_ref = (float *)malloc(size_out);
3235
3236     srand(42);
3237     for (int i = 0; i < row * col; i++)
3238         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3239
3240     // CPU reference: y[j][i] = x[i][j] (row-major)
3241     for (int i = 0; i < row; i++)
3242         for (int j = 0; j < col; j++)
3243             h_y_ref[j * row + i] = h_x[i * col + j];
3244
3245     float *d_x, *d_y;
3246     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3247     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
3248     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3249
3250     dim3 block(16, 16);
3251     dim3 grid((col + 15) / 16, (row + 63) / 64);
3252     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3253     check(cudaGetLastError(), "mattrans_padded launch");
3254     check(cudaDeviceSynchronize(), "mattrans_padded sync");
3255
3256     float *h_y = (float *)malloc(size_out);
3257     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
3258
3259     float max_err = 0.0f;
3260     for (int i = 0; i < col * row; i++) {
3261         float err = fabsf(h_y[i] - h_y_ref[i]);
3262         if (err > max_err) max_err = err;
3263     }
3264     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3265
3266     free(h_x);
3267     free(h_y);
3268     free(h_y_ref);
3269     cudaFree(d_x);
3270     cudaFree(d_y);
3271 }
3272
3273
3274 static void test_sgemv(int M, int K) {
3275

```

```

3276 srand(42);
3277 float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3278 float *h_x = (float *)malloc((size_t)K * sizeof(float));
3279 float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3280 for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3281 for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3282
3283 // CPU reference: y[m] = sum_k A[m,k] * x[k]
3284 for (int m = 0; m < M; m++) {
3285     double sum = 0.0;
3286     for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3287     h_y_ref[m] = (float)sum;
3288 }
3289
3290 float *d_a, *d_x, *d_y;
3291 check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3292 check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3293 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3294
3295 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3296 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3297
3298 dim3 block(32, 4);
3299 dim3 grid((M + 3) / 4);
3300 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3301 check(cudaGetLastError(), "sgemv launch");
3302 check(cudaDeviceSynchronize(), "sgemv sync");
3303
3304 float *h_y = (float *)malloc((size_t)M * sizeof(float));
3305 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3306
3307 float max_err = 0.0f;
3308 for (int m = 0; m < M; m++) {
3309     float err = fabsf(h_y[m] - h_y_ref[m]);
3310     if (err > max_err) max_err = err;
3311 }
3312 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3313
3314 // ---- SGEMV K32 ----
3315 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3316 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3317 check(cudaGetLastError(), "sgemv_k32 launch");
3318 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3319
3320 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
3321
3322 max_err = 0.0f;
3323 for (int m = 0; m < M; m++) {
3324     float err = fabsf(h_y[m] - h_y_ref[m]);
3325     if (err > max_err) max_err = err;
3326 }
3327 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3328
3329 // ---- SGEMV K16 ----
3330 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3331 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3332
3333 int K16 = 16;
3334 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3335 h_x = (float *)malloc((size_t)K16 * sizeof(float));
3336 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3337 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3338 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```

```

3339
3340 for (int m = 0; m < M; m++) {
3341     double sum = 0.0;
3342     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3343     h_y_ref[m] = (float)sum;
3344 }
3345
3346 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3347 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3348 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3349
3350 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3351 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3352
3353 dim3 grid_k16((M + 7) / 8);
3354 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3355 check(cudaGetLastError(), "sgemv_k16 launch");
3356 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3357
3358 h_y = (float *)malloc((size_t)M * sizeof(float));
3359 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3360
3361 max_err = 0.0f;
3362 for (int m = 0; m < M; m++) {
3363     float err = fabsf(h_y[m] - h_y_ref[m]);
3364     if (err > max_err) max_err = err;
3365 }
3366 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3367
3368 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3369 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3370 }
3371
3372
3373 static void test_sgemm(int M, int N, int K) {
3374
3375     size_t size_a = (size_t)M * K * sizeof(float);
3376     size_t size_b = (size_t)K * N * sizeof(float);
3377     size_t size_c = (size_t)M * N * sizeof(float);
3378
3379     float *h_a = (float *)malloc(size_a);
3380     float *h_b = (float *)malloc(size_b);
3381     float *h_c_ref = (float *)malloc(size_c);
3382
3383     srand(42);
3384     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3385     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3386
3387     float *d_a, *d_b, *d_c;
3388     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3389     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3390     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3391
3392     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3393     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3394
3395     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3396     cublasHandle_t handle;
3397     cublasCreate(&handle);
3398     float alpha = 1.0f, beta = 0.0f;
3399     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3400                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
3401     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");

```

```

3402
3403 // Kernel
3404 cudaEvent_t start, stop;
3405 cudaEventCreate(&start);
3406 cudaEventCreate(&stop);
3407
3408 dim3 block(32, 32);
3409 dim3 grid((N + 31) / 32, (M + 31) / 32);
3410 sgemmm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3411 check(cudaGetLastError(), "sgemm launch");
3412 check(cudaDeviceSynchronize(), "sgemm sync");
3413
3414 float *h_c = (float *)malloc(size_c);
3415 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3416
3417 // Verify
3418 float max_err = 0.0f;
3419 for (int i = 0; i < M * N; i++) {
3420     float err = fabsf(h_c[i] - h_c_ref[i]);
3421     if (err > max_err) max_err = err;
3422 }
3423 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3424
3425 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3426
3427 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3428 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3429 sgemmm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3430 check(cudaGetLastError(), "sgemm_vec4 launch");
3431 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3432
3433 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
3434
3435 max_err = 0.0f;
3436 for (int i = 0; i < M * N; i++) {
3437     float err = fabsf(h_c[i] - h_c_ref[i]);
3438     if (err > max_err) max_err = err;
3439 }
3440 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3441
3442 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3443 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3444 cublasDestroy(handle);
3445 cudaEventDestroy(start);
3446 cudaEventDestroy(stop);
3447 }
3448
3449
3450 static void test_hgemmm_mma(int M, int N, int K) {
3451
3452     size_t size_a = (size_t)M * K * sizeof(half);
3453     size_t size_b = (size_t)K * N * sizeof(half);
3454     size_t size_c = (size_t)M * N * sizeof(half);
3455
3456     half *h_a = (half *)malloc(size_a);
3457     half *h_b = (half *)malloc(size_b);
3458     half *h_c_ref = (half *)malloc(size_c);
3459
3460     srand(42);
3461     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3462     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3463
3464     // Kernel expects B^T [NxK] row-major layout (TN layout convention).

```

```

3465 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3466 size_t size_b_t = (size_t)N * K * sizeof(half);
3467 half *h_b_t = (half *)malloc(size_b_t);
3468 for (int n = 0; n < N; n++)
3469     for (int k = 0; k < K; k++)
3470         h_b_t[n * K + k] = h_b[k * N + n];
3471
3472 half *d_a, *d_b, *d_b_t, *d_c;
3473 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
3474 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
3475 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
3476 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
3477
3478 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
3479 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
3480 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
3481
3482 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3483 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3484 cublasHandle_t handle;
3485 cublasCreate(&handle);
3486 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3487 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3488             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3489             &beta_h, d_c, CUDA_R_16F, N,
3490             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3491 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
3492
3493 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3494 cudaEvent_t start, stop;
3495 cudaEventCreate(&start);
3496 cudaEventCreate(&stop);
3497
3498 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3499 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
3500 dim3 block(256);
3501 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3502 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3503 check(cudaGetLastError(), "hgemm launch");
3504 check(cudaDeviceSynchronize(), "hgemm sync");
3505
3506 half *h_c = (half *)malloc(size_c);
3507 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
3508
3509 // Verify
3510 float max_err = 0.0f;
3511 for (int i = 0; i < M * N; i++) {
3512     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3513     if (err > max_err) max_err = err;
3514 }
3515 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3516
3517 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3518 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3519 cublasDestroy(handle);
3520 cudaEventDestroy(start);
3521 cudaEventDestroy(stop);
3522 }
3523
3524
3525 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)
3526 static void test_hgemm_wgmma(int M, int N, int K) {

```

```

3527 // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
3528 // TN layout: C[M×N] = A[M×K] × BT[N×K]
3529 // Kernel: hgemm_wgmma_stages_tn with default template params
3530
3531 constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
3532
3533 // M, K must be divisible by tile dims
3534 if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
3535     printf("| %-35s | %-12s | %-4s |\n",
3536         "HGEMM WGMMMA", "SKIP", "SKIP");
3537     return;
3538 }
3539
3540 size_t size_a = (size_t)M * K * sizeof(half);
3541 size_t size_b = (size_t)K * N * sizeof(half);
3542 size_t size_c = (size_t)M * N * sizeof(half);
3543
3544 half *h_a = (half *)malloc(size_a);
3545 half *h_b = (half *)malloc(size_b);
3546 half *h_c_ref = (half *)malloc(size_c);
3547
3548 srand(42);
3549 for (int i = 0; i < M * K; i++)
3550     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3551 for (int i = 0; i < K * N; i++)
3552     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3553
3554 // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
3555 size_t size_b_t = (size_t)N * K * sizeof(half);
3556 half *h_b_t = (half *)malloc(size_b_t);
3557 for (int n = 0; n < N; n++)
3558     for (int k = 0; k < K; k++)
3559         h_b_t[n * K + k] = h_b[k * N + n];
3560
3561 half *d_a, *d_b, *d_b_t, *d_c;
3562 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
3563 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
3564 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
3565 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
3566
3567 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
3568 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
3569     "wgmma H2D B (cuBLAS)");
3570 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
3571     "wgmma H2D B_t (kernel)");
3572
3573 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
3574 cublasHandle_t handle;
3575 cublasCreate(&handle);
3576 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3577 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
3578     CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
3579     CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3580 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
3581     "wgmma D2H ref");
3582
3583 // Create TMA tensor maps for A and BT
3584 // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
3585 // BT[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
3586 CUTensorMap *tma_a =
3587     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
3588 CUTensorMap *tma_b =
3589     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);

```

```

3590
3591 // Launch WGMMMA kernel
3592 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
3593 size_t smem_bytes =
3594     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
3595 cudaFuncSetAttribute(
3596     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
3597     false>,
3598     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
3599
3600 dim3 block(NUM_THREADS);
3601 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3602 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
3603     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
3604 check(cudaGetLastError(), "wgmma launch");
3605 check(cudaDeviceSynchronize(), "wgmma sync");
3606
3607 half *h_c = (half *)malloc(size_c);
3608 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
3609
3610 // Verify
3611 float max_err = 0.0f;
3612 for (int i = 0; i < M * N; i++) {
3613     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3614     if (err > max_err)
3615         max_err = err;
3616 }
3617 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
3618     max_err < 1.0f ? "PASS" : "FAIL");
3619
3620 free(h_a);
3621 free(h_b);
3622 free(h_b_t);
3623 free(h_c);
3624 free(h_c_ref);
3625 cudaFree(d_a);
3626 cudaFree(d_b);
3627 cudaFree(d_b_t);
3628 cudaFree(d_c);
3629 cudaFree(tma_a);
3630 cudaFree(tma_b);
3631 cublasDestroy(handle);
3632 }
3633 #endif /* NOTES_V2_HAS_WGMMMA */
3634
3635
3636 static void test_flash_attn(int seqlen, int head_dim) {
3637     // FlashAttention-2 with split-Q, MMA m16n8k16
3638     int B = 1, H = 8;
3639
3640     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
3641
3642     srand(42);
3643     half *h_q = (half *)malloc(sz);
3644     half *h_k = (half *)malloc(sz);
3645     half *h_v = (half *)malloc(sz);
3646     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3647         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3648         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3649         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3650     }
3651
3652     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V

```



```

3653 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3654 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3655 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3656 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3657 int count = B * H * seqlen * head_dim;
3658 for (int i = 0; i < count; i++) {
3659     ref_q[i] = __half2float(h_q[i]);
3660     ref_k[i] = __half2float(h_k[i]);
3661     ref_v[i] = __half2float(h_v[i]);
3662 }
3663
3664 float scale = 1.0f / sqrtf((float)head_dim);
3665 for (int bi = 0; bi < B * H; bi++) {
3666     for (int qi = 0; qi < seqlen; qi++) {
3667         // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
3668         float smax = -INFINITY;
3669         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3670         for (int kj = 0; kj < seqlen; kj++) {
3671             float s = 0.0f;
3672             for (int d = 0; d < head_dim; d++)
3673                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3674                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3675             S[kj] = s * scale;
3676             if (S[kj] > smax) smax = S[kj];
3677         }
3678         // softmax
3679         double sum_exp = 0.0;
3680         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3681         float inv_sum = 1.0f / (float)sum_exp;
3682         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
3683         for (int d = 0; d < head_dim; d++) {
3684             double o_acc = 0.0;
3685             for (int kj = 0; kj < seqlen; kj++)
3686                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3687                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
3688             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3689         }
3690         free(S);
3691     }
3692 }
3693
3694 half *d_q, *d_k, *d_v, *d_o;
3695 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3696 check(cudaMalloc(&d_k, sz), "fa alloc K");
3697 check(cudaMalloc(&d_v, sz), "fa alloc V");
3698 check(cudaMalloc(&d_o, sz), "fa alloc O");
3699 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3700 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3701 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3702
3703 // Template params for kHeadDim=64, kStage=2
3704 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3705 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3706 constexpr int kMmaTileSeqLenQ = 4;
3707 constexpr int kMmaTileSeqLenK = 1;
3708 constexpr int kMmaTileSeqLenP = 4;
3709 constexpr int kMmaTileHeadDimV = 1;
3710 constexpr int kValTileSeqLenQ = 1;
3711 constexpr int kValTileSeqLenK = 8;
3712 constexpr int kValTileSeqLenP = 1;
3713 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
3714
3715 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;

```

```

3716 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
3717 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3718                     kStageV * Bc * (kHeadDim + kPadV) +
3719                     Bc * (kHeadDim + kPadV)) * sizeof(half);
3720
3721 dim3 block(128);
3722 dim3 grid((seqlen + Br - 1) / Br, B * H);
3723
3724 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3725 kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3726 kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
3727 kStageV, kPadV>
3728 <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3729 check(cudaGetLastError(), "fa launch");
3730 check(cudaDeviceSynchronize(), "fa sync");
3731
3732 half *h_o = (half *)malloc(sz);
3733 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3734
3735 float max_err = 0.0f;
3736 for (int i = 0; i < count; i++) {
3737     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3738     if (err > max_err) max_err = err;
3739 }
3740 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3741
3742 free(h_q); free(h_k); free(h_v); free(h_o);
3743 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3744 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3745 }
3746
3747
3748 int main(int argc, char *argv[]) {
3749 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
3750     __CUDA_ARCH_FEAT_SM90_ALL)
3751     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
3752 #endif
3753 int M = 1024, N = 1024, K = 1024;
3754 if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
3755
3756 printf("=== notes-v2.cu verification harness ===\n");
3757 printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
3758 printf("|-----|-----|-----|\n");
3759
3760 test_block_reduce(N);
3761 test_dot(N);
3762 test_relu(1024);
3763 test_elementwise(1024);
3764 test_histogram(1024);
3765 test_softmax(256); // softmax kernel requires N == blockDim.x
3766 test_rms_norm(8, 128);
3767 test_layer_norm(8, 128);
3768 test_rope(8, 128);
3769 test_mat_transpose(256, 256);
3770 test_mat_transpose_padded(256, 256);
3771 test_sgemv(256, 128);
3772 test_sgemm(M, N, K);
3773 test_hgemm_mma(M, N, K);
3774 #if (defined(NOTES_V2_HAS_WGMMA) \
3775     || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
3776     || defined(__CUDA_ARCH_FEAT_SM90_ALL))
3777     test_hgemm_wgmma(M, N, K);
3778 #endif

```

```

3778     test_flash_attn(1024, 64);
3779
3780     printf("=== All tests done ===\n");
3781     return 0;
3782 }
3783
3784 // =====
3785 // Quick build & run reference
3786 // =====
3787 // # sm_89 单独编译 + 运行:
3788 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
3789 //
3790 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
3791 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a -DNOTES_V2_HAS_WGMMMA -lcublas -lcuda notes-v2.
    cu -o notes_v2_sm90.bin

```